

Unidad 7

Análisis de componentes principales. Maldición de la dimensionalidad. Matriz de covarianza, autovalores y autovectores. Escalado de datos antes de PCA (Standardization vs. Normalization). Ortogonalidad de las componentes principales. Varianza explicada y varianza acumulada. *Scree Plot* o gráfico de sedimentación. Gráfico de loadings, gráfico de scores, biplot. PCA en el flujo de trabajo de aprendizaje automático. Kernel PCA. Aplicaciones espaciales.

*Reducción de
dimensiones*

La reducción de dimensionalidad comprende varios enfoques que transforman un conjunto de variables (potencialmente muchas) en un número menor de variables que conservan la mayor cantidad posible de la información multidimensional original.

A veces, queremos reducir el número de dimensiones con las que trabajamos en un conjunto de datos para visualizar mejor las relaciones entre ellos o para evitar los fenómenos extraños que se producen en dimensiones elevadas.

Por lo tanto, la reducción de dimensionalidad es una habilidad fundamental que incorporar a tu conjunto de herramientas de aprendizaje automático.

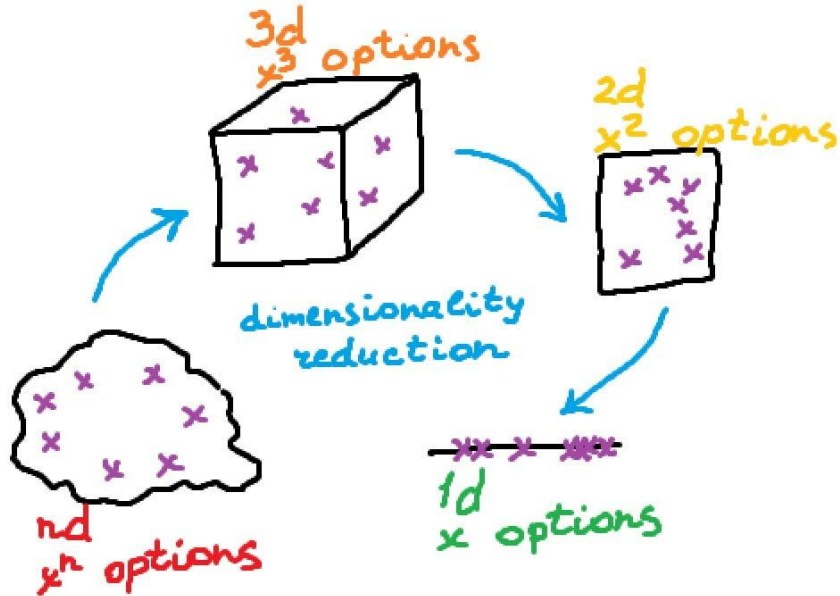
La reducción de dimensiones nos permite:

- Reducir las necesidades de almacenamiento para conjuntos de datos masivos.
- Facilitar la visualización de datos al comprimir la información en menos características.
- Hacer que los modelos de aprendizaje automático sean computacionalmente más eficientes.

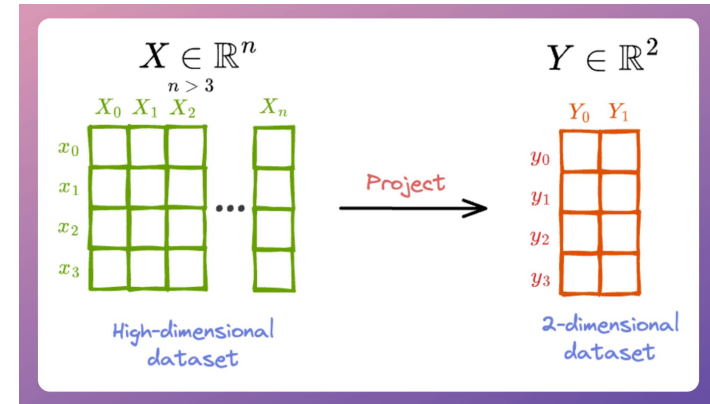
Estos beneficios se traducen en un mejor rendimiento del modelo, mayor interpretabilidad y una mejor escalabilidad de los datos. Pero, por supuesto, la reducción de dimensionalidad conlleva pérdida de datos.

Ninguna técnica de reducción de dimensionalidad es perfecta: por definición, distorsionamos los datos para ajustarlos a dimensiones inferiores.

Reducción de dimensiones



Desafío: ¿Cómo obtener un espacio de menor dimensión (menos columnas) sin perder información relevante de los datos?



Métodos de reducción de dimensiones

- Métodos Lineales

Análisis de Componentes Principales (PCA)

Multidimensional Scaling (MDS)

Análisis de Componentes Independientes (ICA)

- Métodos No lineales

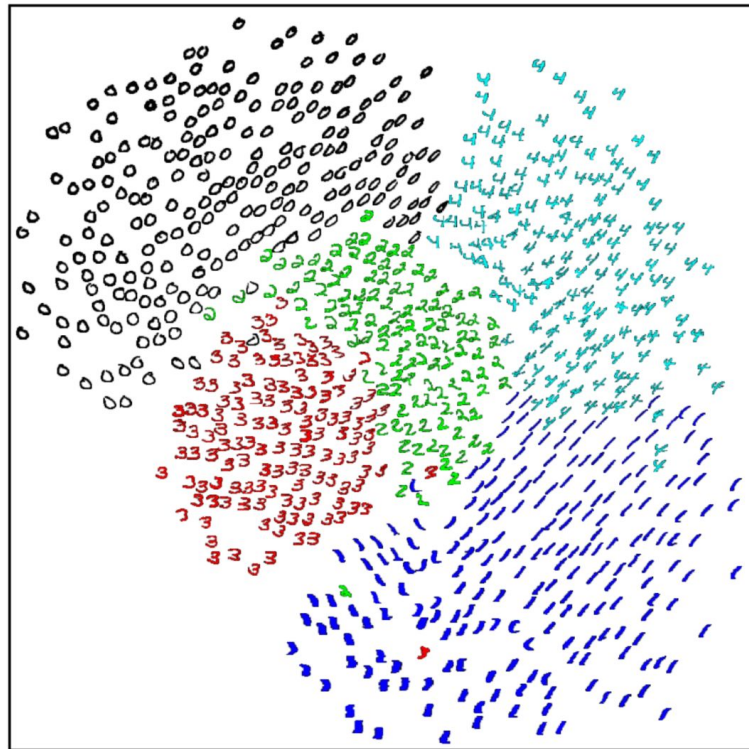
Kernel PCA

t-Distributed Stochastic Neighbour Embeddings (t-SNE)

Uniform Manifold Approximation and Projection (UMAP)

Isometric Feature Mapping (Isomap)

Autoencoders



Análisis de componentes principales

Análisis de componentes principales

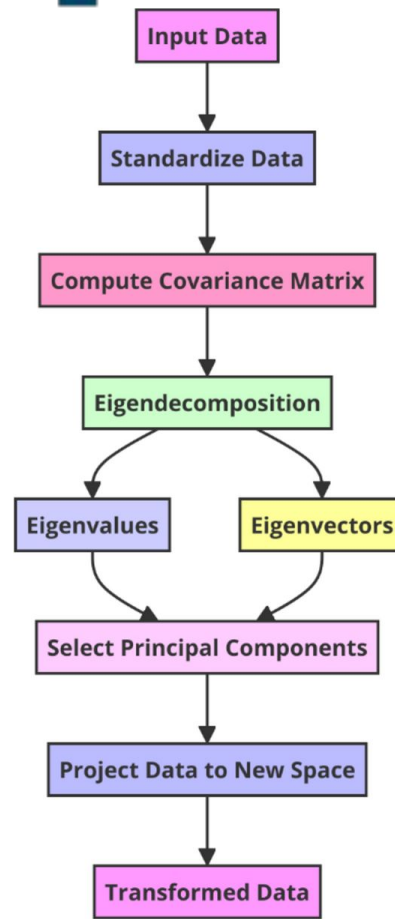
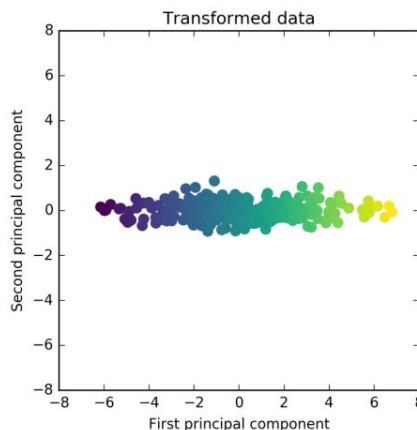
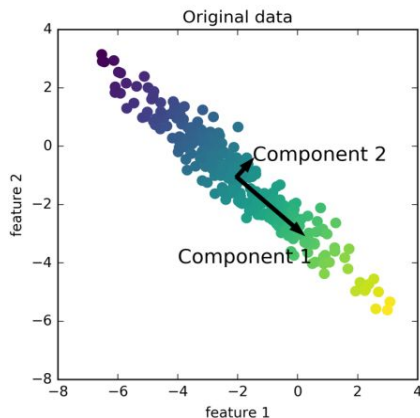
El Análisis de Componentes Principales (PCA, por sus siglas en inglés) constituye una de las metodologías más robustas y versátiles dentro de la estadística multivariante y el aprendizaje automático contemporáneo. Su concepción original, atribuida a Karl Pearson en 1901 y posteriormente desarrollada por Harold Hotelling en 1933, ha evolucionado desde una herramienta de simplificación descriptiva hacia un componente crítico en procesamiento de datos de alta fidelidad.

El propósito fundamental de esta técnica radica en la transformación de un conjunto de variables posiblemente correlacionadas en un nuevo conjunto de variables linealmente no correlacionadas denominadas componentes principales, garantizando que la mayor parte de la variabilidad presente en los datos originales se conserve en las primeras dimensiones de este nuevo espacio.

Análisis de componentes principales

Las Componentes Principales (PCA) es un método no supervisado que rota el dataset de tal forma que **sus ejes X e Y están estadísticamente no correlacionados.**

Esta rotación es seguida por la selección sólo un subset de nuevos features (o variables), de acuerdo a cuán importante son para explicar los datos.



En la práctica, es común enfrentarse a conjuntos de datos donde el número de características o variables (p) es comparable o incluso superior al número de observaciones (n). Este fenómeno introduce el problema crítico conocido como la "maldición de la dimensionalidad". A medida que el número de dimensiones aumenta, el volumen del espacio se expande de forma exponencial, lo que provoca que los puntos de datos se vuelvan extremadamente dispersos. Esta dispersión tiene implicaciones profundas para cualquier análisis estadístico o modelo de predicción: en espacios de alta dimensión, el concepto de proximidad se desvanece, ya que la diferencia entre la distancia al vecino más cercano y la distancia al vecino más lejano tiende a cero.

La maldición de la dimensionalidad no solo afecta la interpretación, sino que degrada el rendimiento de los modelos de aprendizaje automático. Un exceso de características puede inducir al sobreajuste (overfitting), donde el modelo captura el ruido aleatorio en lugar de la señal estructural subyacente.

Maldición de la dimensionalidad

Consecuencia de la dimensionalidad	Impacto en el modelado de datos	Relación con el PCA
Dispersión de datos	Las muestras se vuelven insuficientes para representar patrones globales.	El PCA identifica los ejes de mayor densidad para concentrar la información.
Inestabilidad de parámetros	Aumenta la varianza de los estimadores, reduciendo la precisión.	La reducción de variables irrelevantes mejora la estabilidad del modelo.
Carga computacional	El tiempo de procesamiento y el uso de memoria crecen drásticamente.	La proyección a bajas dimensiones acelera el entrenamiento y la inferencia.
Visualización	Imposibilidad de inspeccionar gráficamente relaciones entre variables.	Permite proyecciones en 2D o 3D para análisis exploratorio.

- Centrado

Restar la media a cada variable para que tengan media 0. Esto es **obligatorio** en PCA, porque el algoritmo calcula varianzas y covarianzas respecto al origen. Sin centrar, el primer componente principal apuntaría hacia la media de los datos en lugar de la dirección de máxima varianza.

- Escalado

Dividir por el desvío estándar para que cada variable tenga varianza 1. Se recomienda cuando:

- Las variables tienen **distintas unidades** (ej: altura en cm y peso en kg)
- Las variables tienen **magnitudes muy diferentes**

Si no se escala, PCA va a priorizar las variables con mayor varianza simplemente por tener valores más grandes, no por ser más informativas.

Centrado

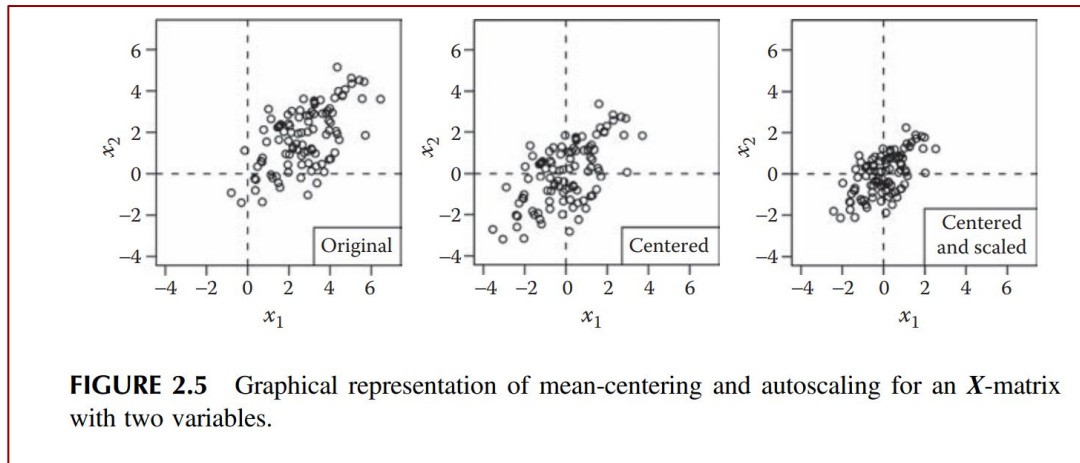
Dada una matriz de datos $X \in \mathbb{R}^{n \times p}$, donde n es el número de observaciones y p el número de variables, el centrado consiste en transformar cada variable j restando su media muestral:

$$x_{ij} = x_i - x_j$$

de modo que X satisfice:

$$\text{donde } x_j = 1/n \sum x_{ij}$$

$$1/n \sum x_{ij} = 0 \quad \forall j=1, \dots, p.$$



El escalado es el procedimiento estándar para el PCA. Este proceso garantiza que todas las variables contribuyan de manera equitativa a la formación de las componentes principales, permitiendo que la estructura de correlación guíe la transformación en lugar de la magnitud absoluta de las cifras. La estandarización es particularmente necesaria cuando las variables poseen diferentes unidades físicas (kilómetros, grados, dólares) o cuando no comparten el mismo orden de magnitud.

Es importante escalar los datos antes de usar métodos basados en distancias, como Support Vector Machines o K nearest neighbors.

Escalando, las variables pasan a ser pesadas de la misma forma.

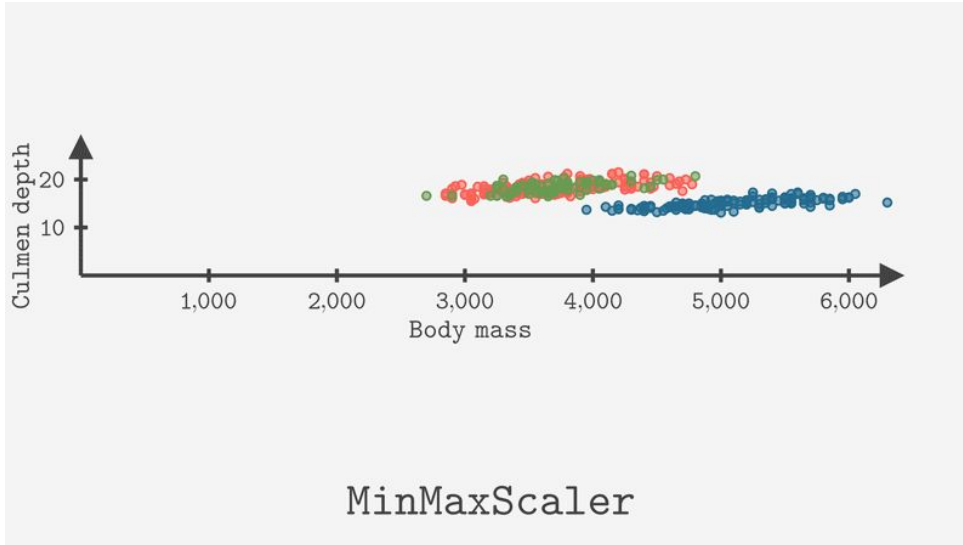
En scikit-learn tenemos como opciones:

- MinMaxScaler <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>
- MaxAbsScaler <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MaxAbsScaler.html>
- StandardScaler <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

MinMaxScaler

MinMaxScaler: escala las variables a un rango entre 0 y 1

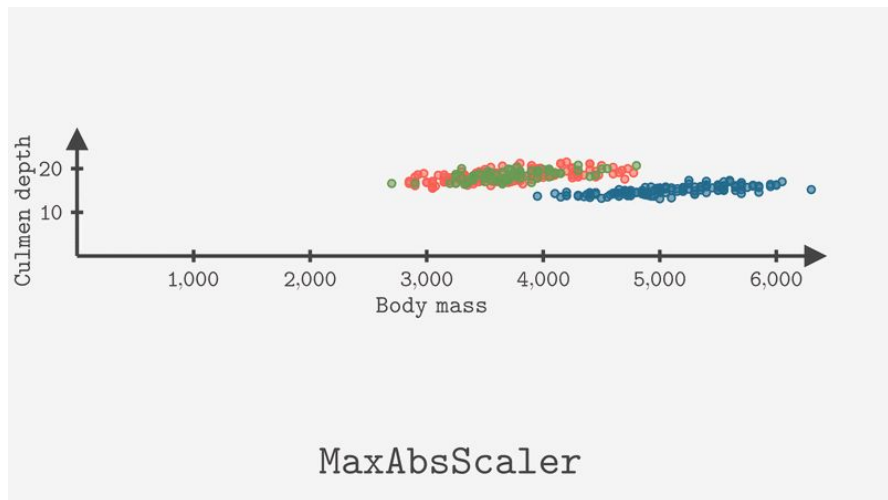
$$\text{MinMaxScaler: } x_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$



MaxAbsScaler

MaxAbsScaler: escala las variables de tal manera que el valor absoluto máximo sea 1 (por lo que se garantiza que los datos queden en un rango entre -1 y 1)

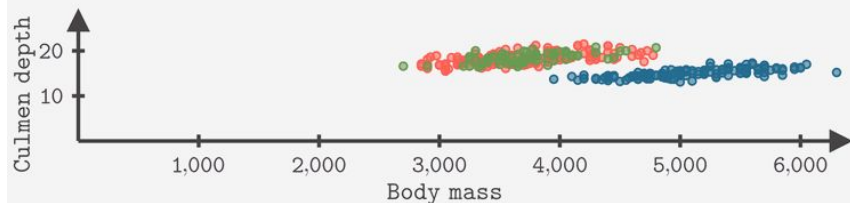
$$\text{MaxAbsScaler} : x_{\text{scaled}} = \frac{x}{\max|x|}$$



StandardScaler

StandardScaler: estandariza las variables haciendo que la media sea igual a 0 y la varianza sea igual a 1.

$$\text{StandardScaler} : x_{\text{scaled}} = \frac{x - \bar{x}}{\sigma}$$



StandardScaler

La que vamos a utilizar para
PCA

Matriz de varianzas y covarianzas

La matriz de covarianza σ (sigma) es una representación cuadrada y simétrica de tamaño d donde cada elemento (i, j) indica la covarianza entre la variable i y la variable j . La diagonal de la matriz contiene las varianzas de cada variable individual. Una covarianza positiva alta indica que las variables tienden a aumentar juntas, lo que sugiere una redundancia informativa que el PCA puede explotar para comprimir el espacio.

El cálculo de la matriz de covarianza para un conjunto de datos X (centrado) se define como:

$$\sigma = 1 / (n-1) X^T X$$

$$\Sigma = \begin{pmatrix} \sigma_{11}^2 & \sigma_{12} & \dots & \sigma_{1m} \\ \sigma_{21} & \sigma_{22}^2 & \dots & \sigma_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \dots & \sigma_{nm}^2 \end{pmatrix}$$

la diagonal principal es la
varianza

$$\Sigma = \begin{pmatrix} \sigma_{11}^2 & \sigma_{12} & \dots & \sigma_{1m} \\ \sigma_{21} & \sigma_{22}^2 & \dots & \sigma_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \dots & \sigma_{nm}^2 \end{pmatrix}$$

por fuera de la diagonal principal
tenemos las **covarianzas**

- Los eigenvectors son un caso particular de multiplicación entre una matriz y un vector. Obsérvese la siguiente multiplicación

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} x \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 8 \end{pmatrix} = 4 x \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

El vector resultante de la multiplicación es un múltiplo entero del vector original. Los eigenvectors de una matriz son todos aquellos vectores que, al multiplicarlos por dicha matriz, resultan en el mismo vector o en un múltiplo entero del mismo. Los *eigenvectors* tienen una serie de propiedades matemáticas específicas:

- Los eigenvectors solo existen para matrices cuadradas y no para todas. En el caso de que una matriz $n \times n$ tenga *eigenvectors*, el número de ellos es n .
- Si se escala un *eigenvector* antes de multiplicarlo por la matriz, se obtiene un múltiplo del mismo *eigenvector*. Esto se debe a que si se escala un vector multiplicándolo por cierta cantidad, lo único que se consigue es cambiar su longitud pero la dirección es la misma.
- Todos los *eigenvectors* de una matriz son perpendiculares (ortogonales) entre ellos, independientemente de las dimensiones que tengan.

Cuando se multiplica una matriz por alguno de sus *eigenvectors* se obtiene un múltiplo del vector original, es decir, el resultado es ese mismo vector multiplicado por un número. **Al valor por el que se multiplica el *eigenvector* resultante se le conoce como eigenvalue.** A todo *eigenvector* le corresponde un *eigenvalue* y viceversa.

En el método PCA, cada una de las componentes se corresponde con un *eigenvector*, y el orden de componente se establece por orden decreciente de *eigenvalue*. Así pues, el primer componente es el *eigenvector* con el *eigenvalue* asociado más alto.

Cálculo de las componentes principales

Dado un set de datos X con n observaciones y p variables, el proceso a seguir para calcular la primera componente principal es:

- **Centralización de las variables**: se resta a cada valor la media de la variable a la que pertenece. Con esto se consigue que todas las variables tengan media cero.
- Se resuelve un problema de optimización para encontrar el valor de los *loadings* con los que se maximiza la varianza. Una forma de resolver esta optimización es mediante el **cálculo de eigenvector-eigenvalue de la matriz de covarianzas**.

Una vez calculada la primera componente (Z_1) se calcula la (Z_2) repitiendo el mismo proceso, pero añadiendo la condición de que la combinación lineal no puede estar correlacionada con la primera componente. Es lo mismo decir que Z_1 y Z_2 son perpendiculares.

El proceso se repite hasta tener todas las componentes principales o hasta que se decida detener el proceso. El orden de importancia de las componentes viene dado por la magnitud del eigenvalue asociado a cada eigenvector.

Maximización de la varianza

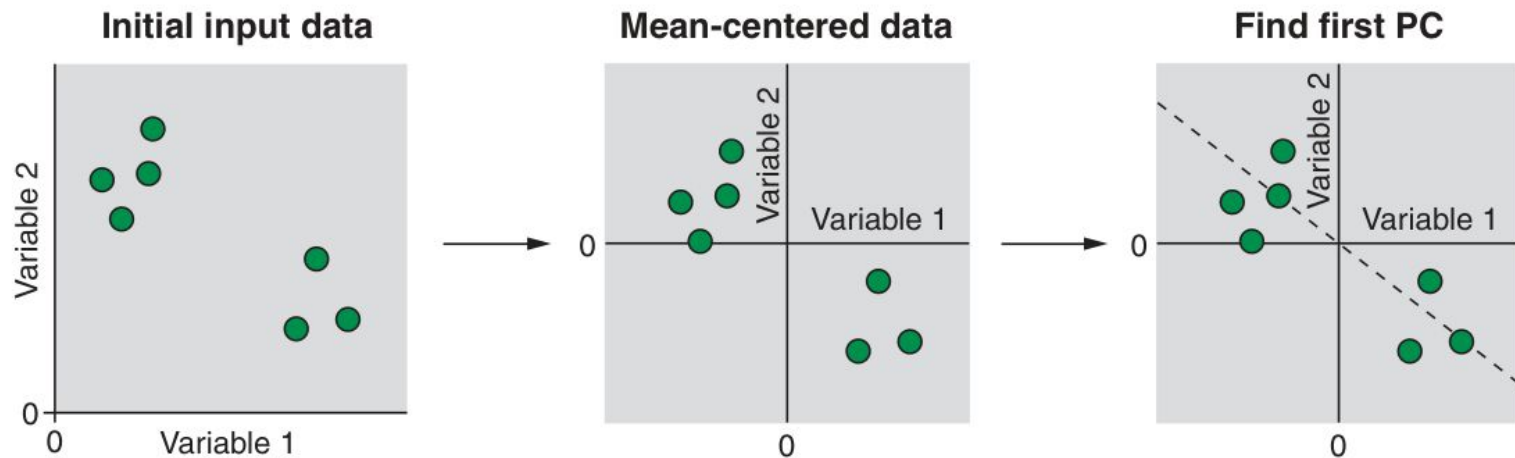


Figure 13.2 The first thing we do before applying the PCA algorithm is (usually) to center the data by subtracting the mean of each variable for each case. This places the origin at the center of the data. The first principal axis is then found: it is the axis that passes through the origin and maximizes the variance of the data when projected onto it.

Maximización de la varianza

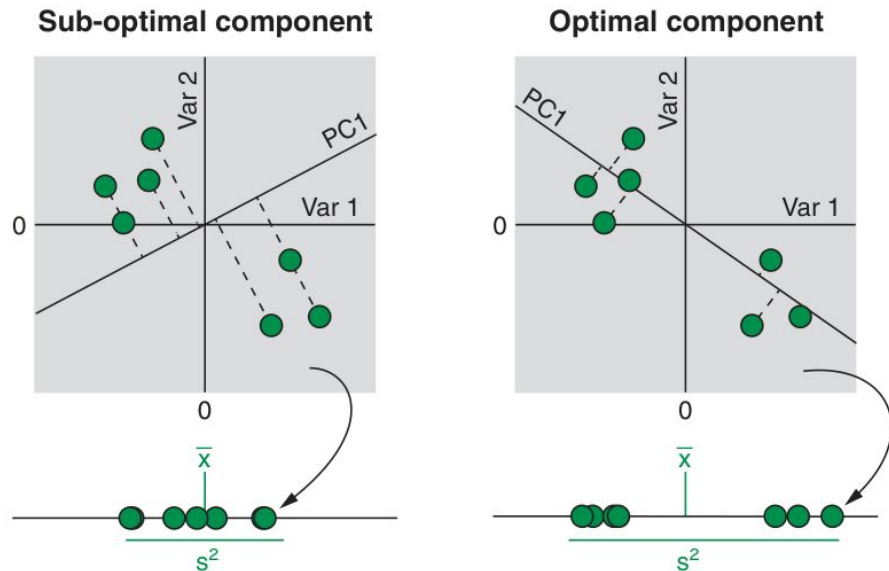


Figure 13.3 What it means for the first principal axis to “maximize the variance.” The left-side plot shows a sub-optimal candidate principal axis. The right-side plot shows the optimal candidate principal axis. The data is shown projected onto each principal axis below the respective plots. The variance of the data along the axis is greatest on the right side.

Maximización de la varianza

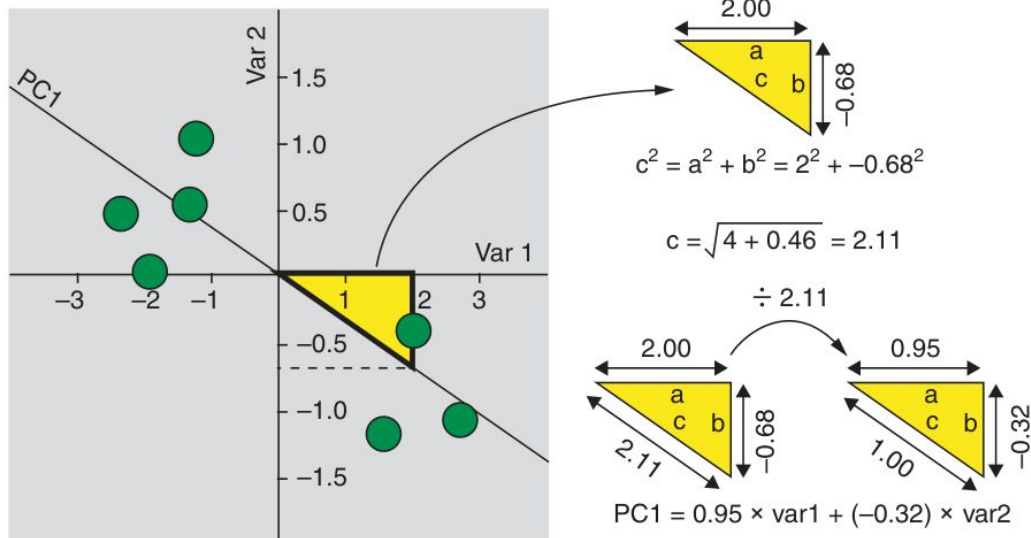
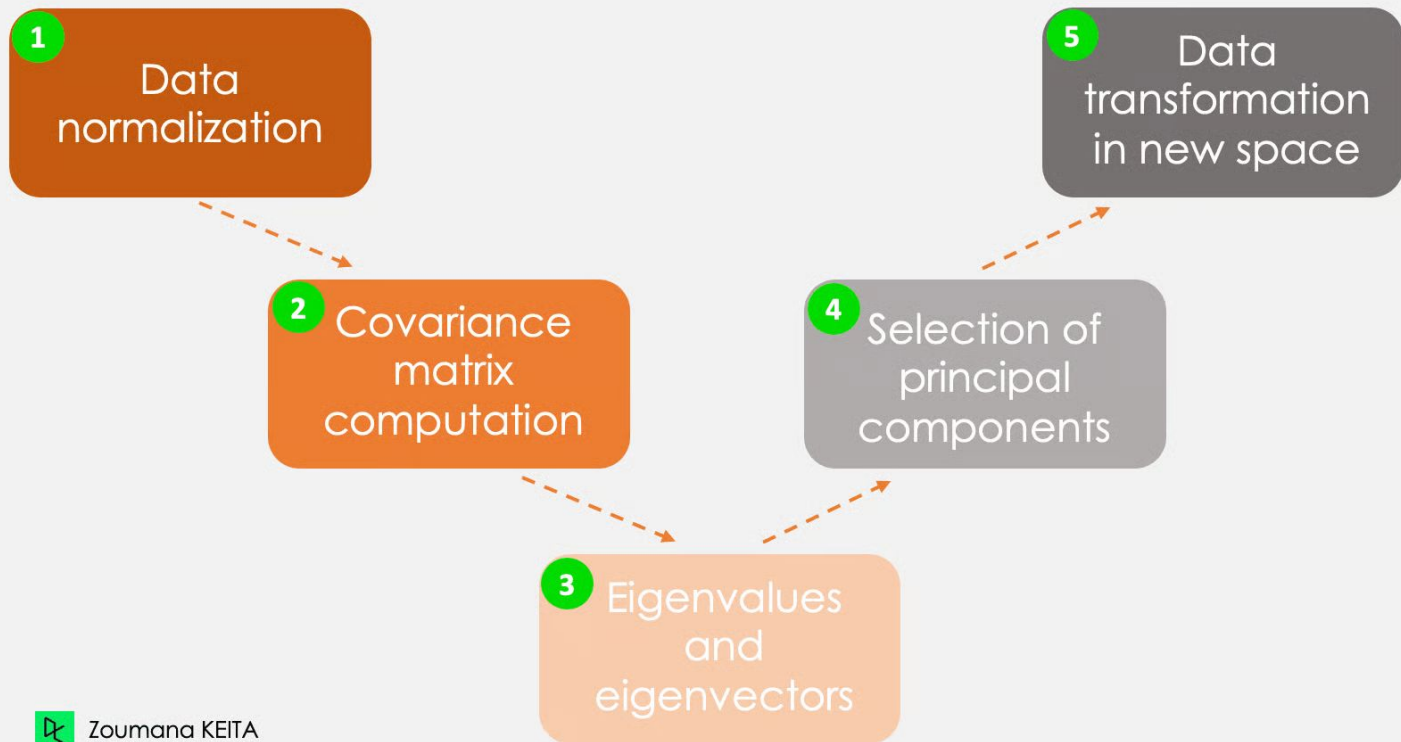


Figure 13.4 Calculating the eigenvector for a principal component. The distances along each variable are scaled so that they mark a point that is one unit along the principal axis away from the origin. We can illustrate this graphically by taking a triangle defined by the change in one variable over the change in the other variable, and using Pythagoras's theorem to find the distance from the origin to divide by.

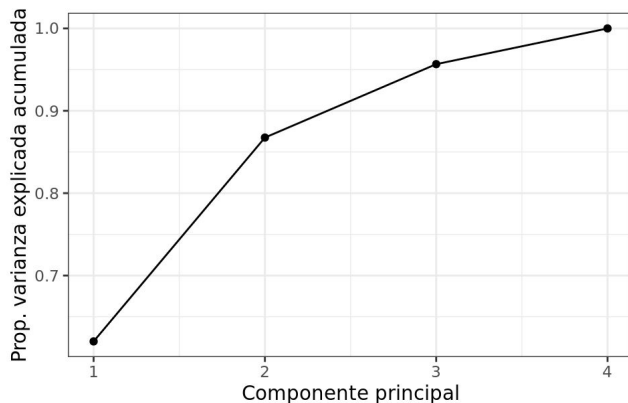
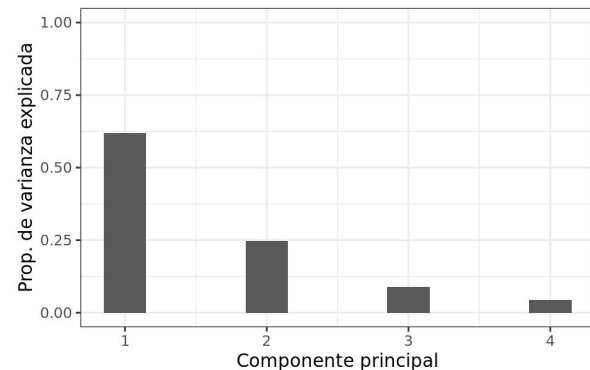
Pasos de PCA



Varianza explicada y varianza acumulada

Una de las preguntas más frecuentes que surge tras realizar un PCA es: ¿Cuánta información presente en el set de datos original se pierde al proyectar las observaciones en un espacio de menor dimensión? o lo que es lo mismo ¿Cuánta información es capaz de capturar cada una de las componentes principales obtenidas? Para contestar a estas preguntas se recurre a la proporción de varianza explicada por cada componente principal.

Tanto la proporción de varianza explicada como la proporción de varianza explicada acumulada son dos valores de gran utilidad a la hora de decidir el número de componentes principales a utilizar en los análisis posteriores. Si se calculan todas las componentes principales de un set de datos, entonces, aunque transformada, se está almacenando toda la información presente en los datos originales. El sumatorio de la proporción de varianza explicada acumulada de todas las componentes es siempre 1.



Scree Plot o gráfico de sedimentación

El Scree Plot es la herramienta visual primaria para determinar el número óptimo de componentes. Consiste en un gráfico de líneas que muestra los autovalores o el porcentaje de varianza explicada frente al número de la componente. El objetivo es identificar el "codo" de la gráfica, el punto donde la magnitud de los autovalores cae abruptamente y comienza a estabilizarse en valores bajos. Los componentes antes del codo se consideran portadores de la estructura real de los datos, mientras que los posteriores suelen representar ruido.

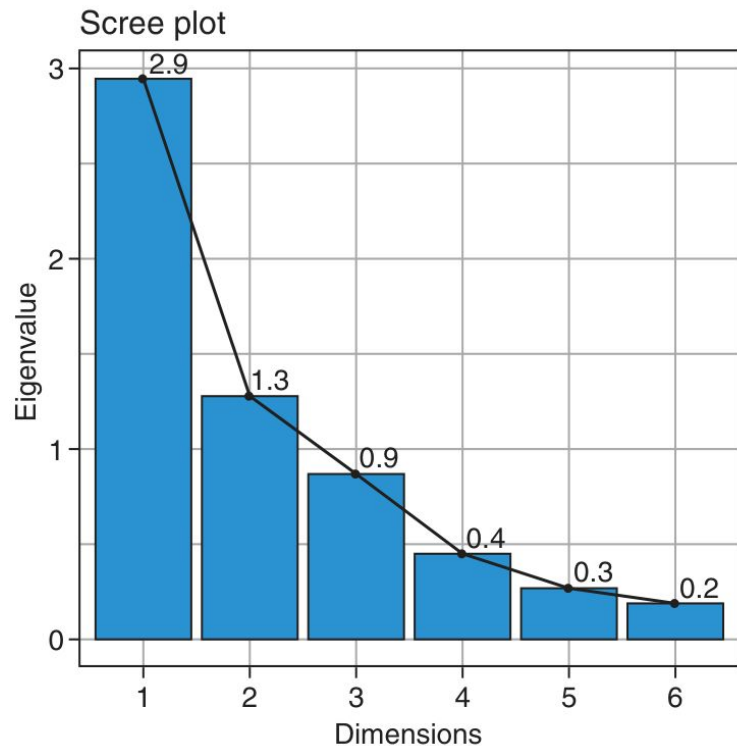


Gráfico de loadings

Las cargas o loadings son los pesos que cada variable original aporta a una componente principal específica. Representan la correlación entre las variables originales y los nuevos ejes. En una visualización de loadings, las variables se representan como vectores:

- La dirección del vector indica si la variable contribuye positiva o negativamente a la componente.
- La longitud del vector (magnitud) refleja la importancia de esa variable en el plano seleccionado; vectores largos indican una fuerte influencia.
- La proximidad angular entre vectores revela correlaciones entre las variables originales: ángulos agudos indican correlación positiva, ángulos de 90° indican independencia y ángulos de 180° indican correlación negativa.

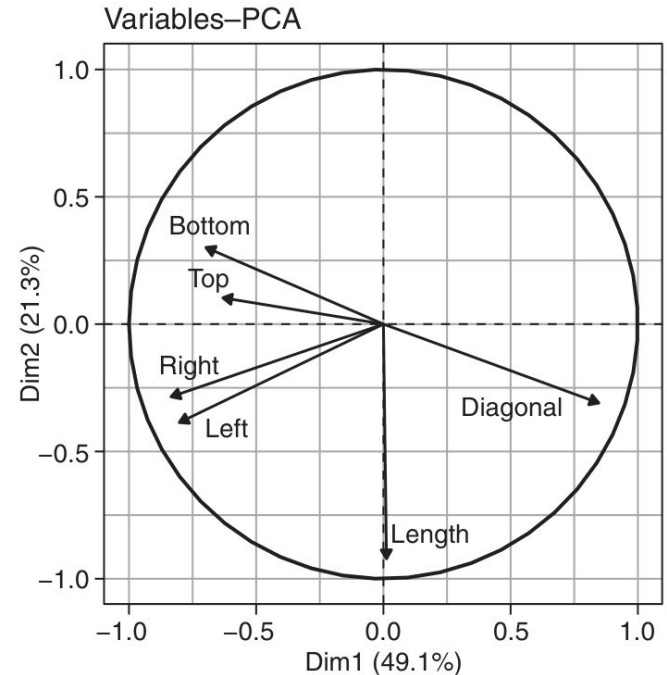
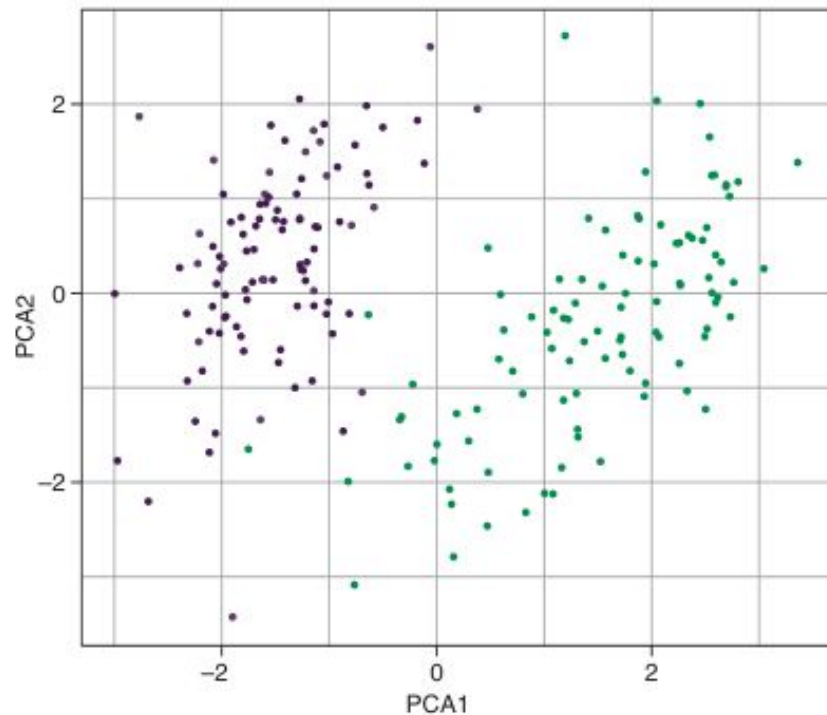


Gráfico de scores

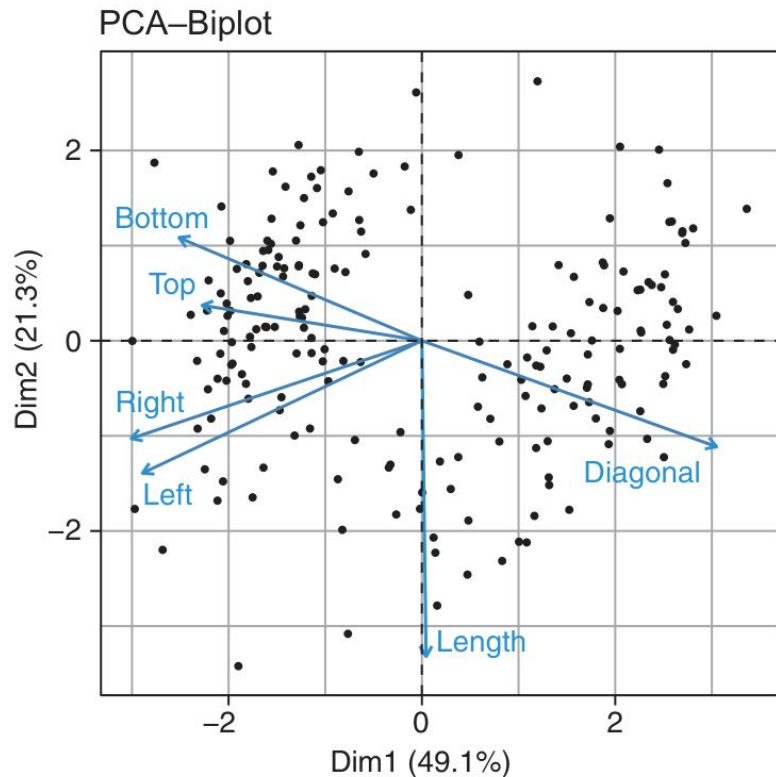
Los scores representan las coordenadas de las observaciones originales en el nuevo sistema de componentes principales. Al graficar la PC1 contra la PC2, se obtiene un mapa de dispersión que revela la estructura intrínseca de los datos.

Las observaciones que aparecen cerca unas de otras en este gráfico comparten características similares en sus variables originales, lo que permite identificar grupos naturales (clusters) o valores atípicos de manera intuitiva.



Biplot

El Biplot es quizás la herramienta de visualización más poderosa del PCA, ya que superpone el gráfico de scores y el de loadings en un solo plano. Esto permite una interpretación directa de por qué ciertas observaciones se agrupan en determinadas regiones del espacio. Si un grupo de muestras se sitúa en la misma dirección hacia la que apunta el vector de una variable, se puede inferir que ese grupo de muestras tiene valores altos en dicha variable. El biplot transforma los números abstractos del PCA en un mapa perceptual que facilita la toma de decisiones estratégicas.



PCA en el flujo de aprendizaje automático

El PCA no es solo una técnica exploratoria, sino un paso fundamental en el preprocesamiento para modelos predictivos. Su integración en tuberías (pipelines) automatizadas ofrece ventajas competitivas significativas.

1. **Eliminación de redundancia:** Al proyectar los datos sobre ejes ortogonales, el PCA aborda la multicolinealidad, lo que mejora la interpretabilidad de los coeficientes en modelos lineales y la estabilidad numérica de los algoritmos de optimización como el descenso de gradiente.
2. **Filtrado de ruido:** Al retener solo las componentes con mayor varianza, se descartan las dimensiones que contienen principalmente errores de medición o fluctuaciones aleatorias, lo que a menudo resulta en una mejor generalización del modelo en datos no vistos.
3. **Compresión de datos:** Reducir un conjunto de 1,000 variables a 50 componentes principales disminuye drásticamente los requisitos de memoria y acelera la velocidad de entrenamiento e inferencia, lo cual es crítico en aplicaciones de tiempo real o sistemas embebidos.

Ventajas y Desventajas de PCA



VENTAJAS

Reducción de Dimensionalidad: Permite comprimir el dataset manteniendo la mayor cantidad de información (varianza) posible. Esto es vital para algoritmos que sufren la "maldición de la dimensionalidad".

Eliminación de Multicolinealidad: PCA transforma variables correlacionadas en un conjunto de variables ortogonales (independientes entre sí), lo que mejora el rendimiento de modelos de regresión lineal o logística.

Visualización: Al reducir los datos a 2 o 3 componentes principales, puedes graficar nubes de puntos que antes eran imposibles de visualizar, permitiendo identificar clusters o patrones a simple vista.

Reducción de Ruido: Al descartar los componentes con menor varianza, a menudo se está eliminando "ruido" aleatorio del dataset, quedándote con la señal principal.

Eficiencia Computacional: Menos variables significan entrenamientos más rápidos y menor consumo de memoria.

DESVENTAJAS

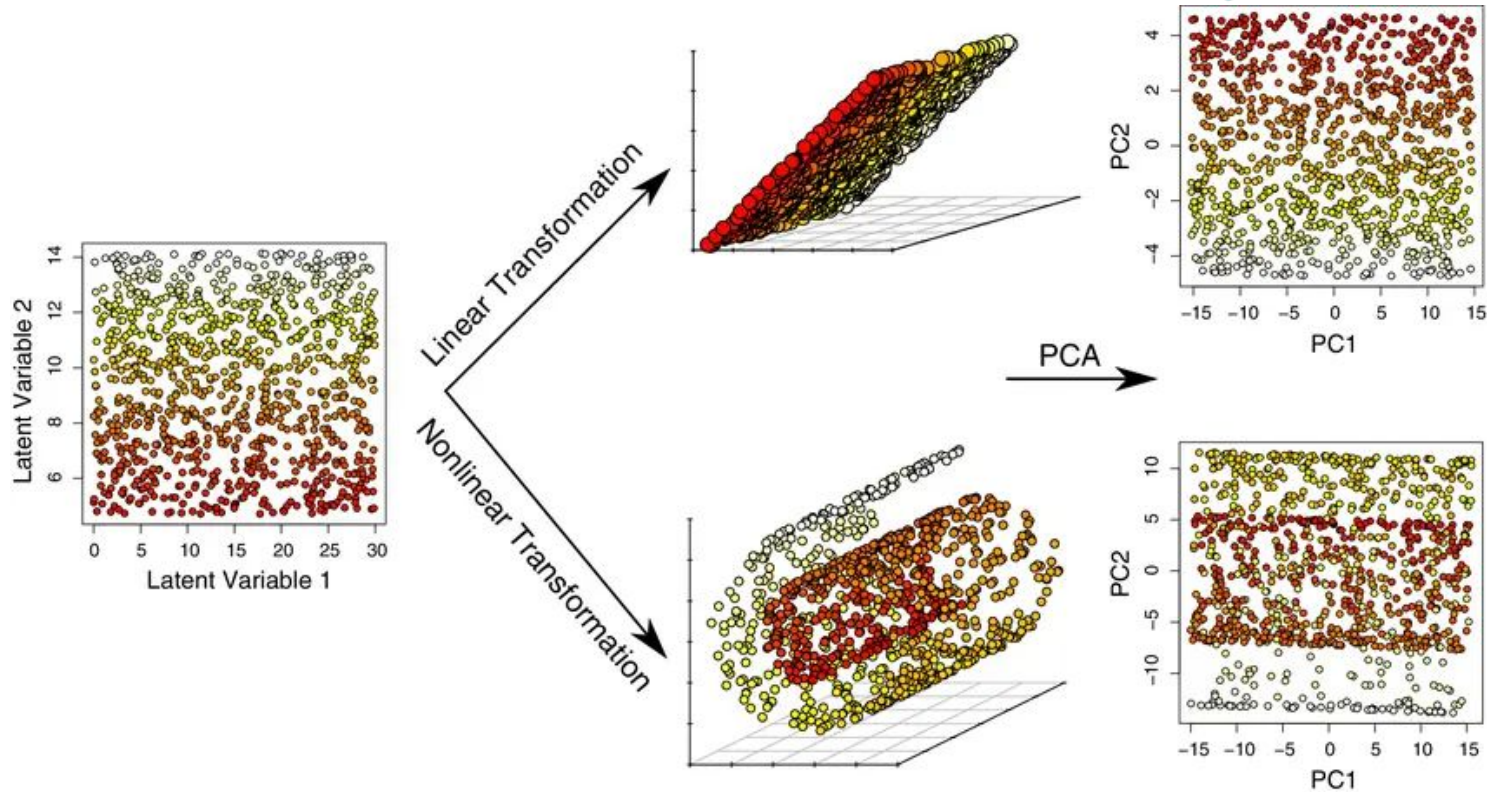
Pérdida de Interpretabilidad: Esta es la mayor desventaja. Las nuevas variables (componentes) son combinaciones lineales de las originales. Ya no puedes decir "la edad afecta al resultado"; ahora tienes un "Componente 1" que es una mezcla matemática de edad, ingresos y altura.

Sensibilidad a la Escala: PCA se basa en la varianza. Si una variable tiene números muy grandes (ej. salarios) y otra muy pequeños (ej. años de estudio), la de números grandes dominará el análisis. **Siempre hay que estandarizar los datos antes.**

Suposición de Linealidad: PCA busca relaciones lineales. Si tus datos tienen estructuras complejas no lineales (como una espiral), PCA no será efectivo (en esos casos se usa *Kernel PCA*).

Pérdida de Información: Por mucho que intentes retener la varianza, siempre hay una pérdida de información al descartar componentes. Si el dataset es muy complejo, esa pérdida puede ser crítica.

Transformaciones



The strengths of PCA are as follows:

- PCA creates new axes that are directly interpretable in terms of the original variables.
- New data can be projected onto the principal axes.
- PCA is really a mathematical transformation and so is computationally inexpensive.

The weaknesses of PCA are these:

- Mapping from high dimensions to low dimensions cannot be nonlinear.
- It cannot handle categorical variables natively.
- The final number of principal components to retain must be decided by us for the application at hand.

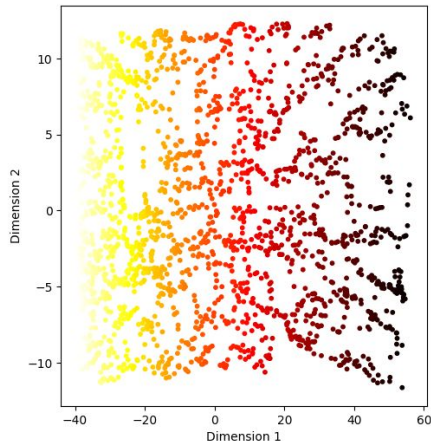
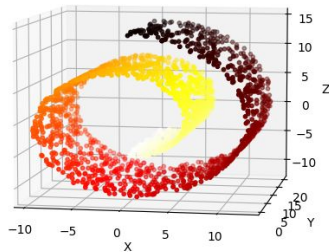
- Preprocesamiento para aprendizaje automático
- Visualización de datos de alta dimensión
- Compresión de imágenes y señales
- Bioinformática y genética
- Finanzas y análisis de riesgo

Kernel PCA

Kernel PCA

El PCA tradicional es una técnica lineal; asume que el subespacio que describe los datos es un plano o hiperplano. Sin embargo, en la naturaleza y en la industria, los datos a menudo se encuentran en "variedades" (manifolds) curvas. Un ejemplo clásico es el "Swiss Roll", un conjunto de datos tridimensional que se enrolla sobre sí mismo. El PCA lineal fallaría al intentar "desenrollar" estos datos, ya que solo puede ver proyecciones planas.

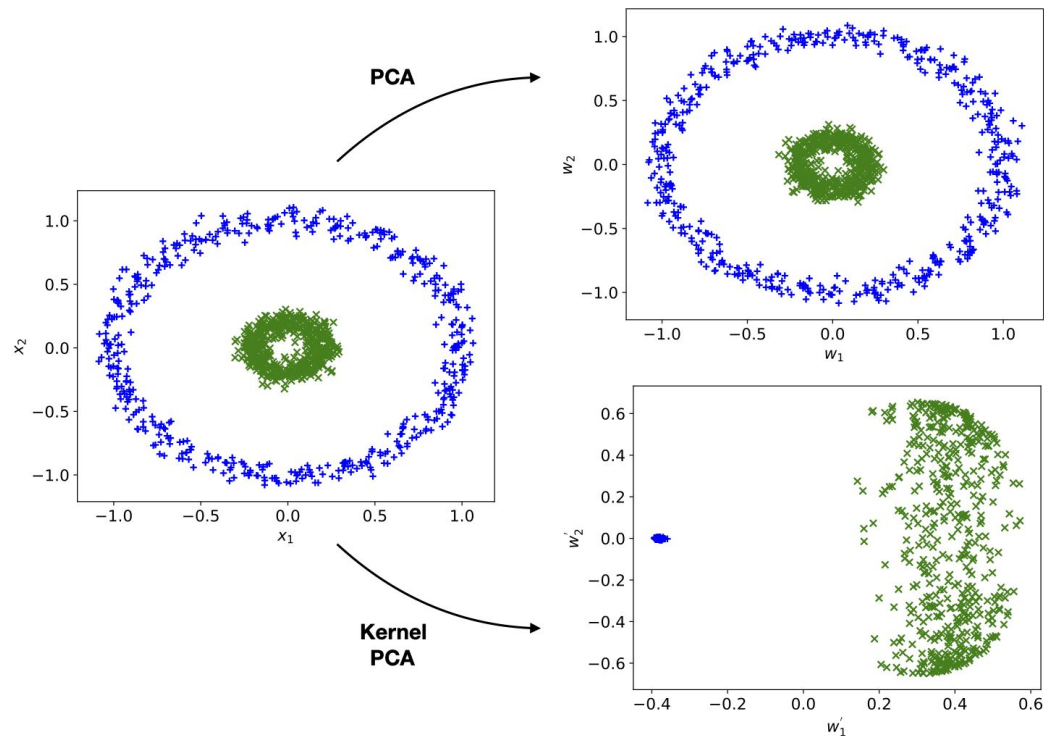
El Kernel PCA (KPCA) resuelve esta limitación mediante el "truco del kernel". En lugar de trabajar en el espacio original, el KPCA proyecta implícitamente los datos a un espacio de características de dimensión mucho mayor (o infinita) donde la estructura no lineal se vuelve linealmente separable. Una vez en este espacio de alta dimensión, se aplica el PCA estándar.



Kernel PCA

En lugar de buscar componentes en el espacio original de dos dimensiones, el Kernel PCA proyecta implícitamente los datos a un espacio de características de mayor dimensión (un espacio abstracto de dimensiones infinitas o muy altas) mediante una función matemática llamada **Kernel** (probablemente un Kernel de Base Radial o RBF en este caso).

El Kernel "levanta" el círculo verde hacia arriba en una tercera dimensión z , creando un cono. Si se pasa un cuchillo (un hiperplano lineal) de forma horizontal, se separa el cono de la base.



Los kernels más utilizados incluyen:

- **Kernel RBF (Radial Basis Function):** Excelente para detectar estructuras locales y formas complejas no circulares.
- **Kernel Polinómico:** Útil para datos donde las relaciones dependen de interacciones de orden superior entre variables.
- **Kernel Sigmoide:** Proviene de la literatura de redes neuronales y es útil en ciertos contextos de clasificación.

t-SNE

t-SNE (t-Distributed Stochastic Neighbor Embedding) se define como un algoritmo de aprendizaje automático de **reducción de dimensionalidad no lineal**, cuyo propósito principal es la visualización topológica de conjuntos de datos de alta dimensión proyectándolos en un espacio de baja dimensión (típicamente 2D o 3D).

Su fundamento matemático radica en preservar las estructuras locales de los datos, garantizando que los objetos similares en el espacio original sean modelados por puntos muy cercanos en la proyección final, mientras que los objetos disimilares sean empujados a una mayor distancia.

Casos ideales para t-SNE:

- **Relaciones no lineales:** Cuando tus datos tienen patrones curvos, intrincados o muy complejos.
- **Enfoque en la estructura local:** Si tu objetivo principal es agrupar elementos muy similares (por ejemplo, descubrir subtipos de células muy parecidas entre sí), t-SNE es fantástico porque está diseñado para mantener juntos a los vecinos cercanos y empujar lejos a los distintos.
- **Conjuntos de datos pequeños a medianos:** Debido a que t-SNE es computacionalmente costoso y lento, es preferible usarlo en conjuntos que no sean masivamente grandes.

Formalmente, el algoritmo opera mediante un proceso que modela las relaciones geométricas como distribuciones de probabilidad a través de los siguientes pasos:

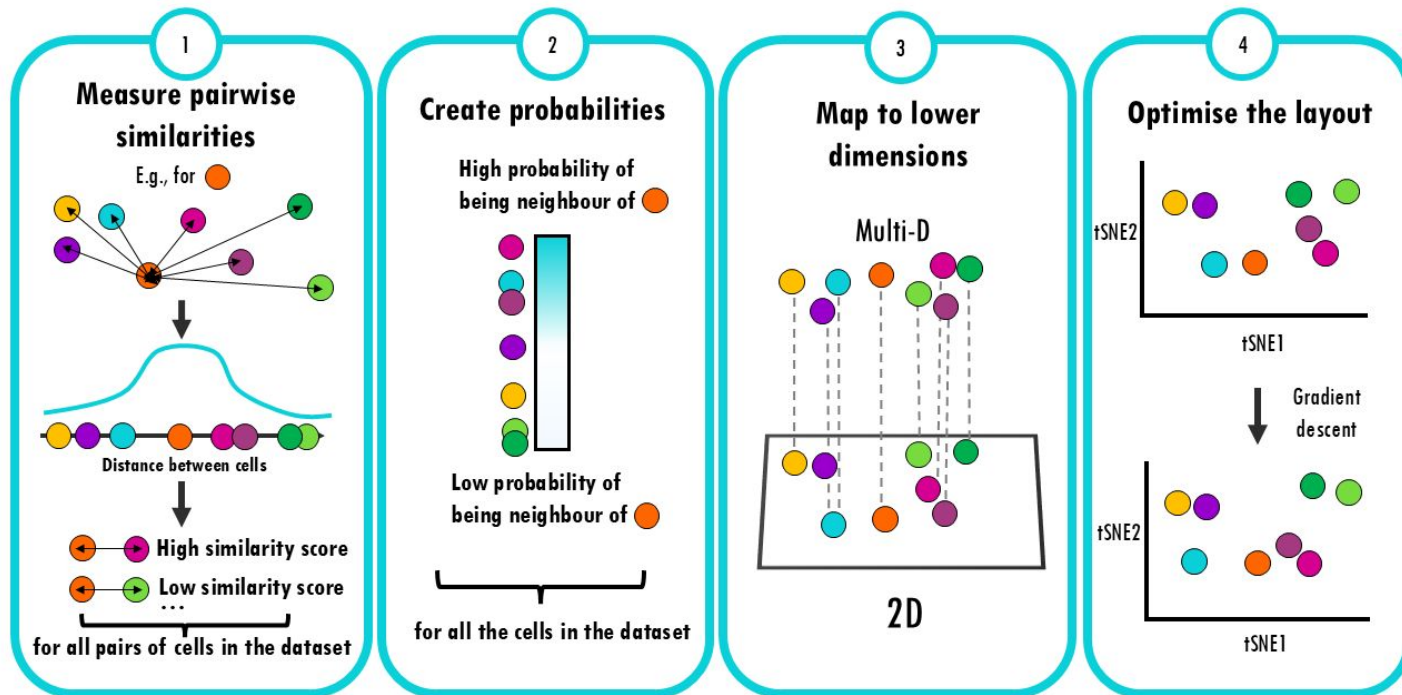
1. **Modelado de similitudes en el espacio de alta dimensión:** Para cada punto en el espacio original, t -SNE calcula la probabilidad de que otros puntos sean sus vecinos. Esta similitud por pares se calcula midiendo la distancia entre los puntos y evaluándola bajo una **distribución normal (Gaussiana)** centrada en el punto de interés. Estas distancias se normalizan (escalan para que sumen 1), construyendo así una matriz de probabilidad que representa las afinidades vecinales en el espacio original.
2. **Modelado de similitudes en el espacio de baja dimensión (El uso de la distribución t):** Posteriormente, el algoritmo crea una proyección aleatoria inicial de los datos en el espacio reducido. Para medir las similitudes entre los puntos en esta nueva dimensión, t -SNE abandona la distribución normal y utiliza una **distribución t de Student**. Matemáticamente, la distribución t tiene un pico más bajo y colas más gruesas, lo cual es la clave del algoritmo para resolver el "problema de aglomeración" (*crowding problem*); esto le otorga a los puntos disimilares más espacio para separarse en las dimensiones bajas en lugar de colapsar todos en el centro del plano.

3. Optimización de la proyección (Función de costo): El objetivo final del algoritmo es lograr que la matriz de probabilidades del espacio reducido sea idéntica (o lo más cercana posible) a la matriz de probabilidades del espacio de alta dimensión. Para medir la diferencia entre estas dos distribuciones de probabilidad, t-SNE utiliza una métrica matemática llamada **divergencia de Kullback-Leibler (KL divergence)**. A través de un método de optimización iterativo conocido como **descenso de gradiente** (*gradient descent*), el algoritmo ajusta paso a paso las posiciones de los puntos en el mapa 2D minimizando esta divergencia de Kullback-Leibler, estabilizándose cuando la representación de baja dimensión refleja fielmente la original.

Pasos de t-SNE

Paso	Qué se realiza
1	Calcular similitudes en alta dimensión con Gaussiana
2	Inicializar puntos en baja dimensión
3	Calcular similitudes en baja dimensión con distribución t
4	Definir función de costo (KL divergence)
5	Optimización con el descenso del gradiente

How does t -SNE summarise many dimensions into 2?



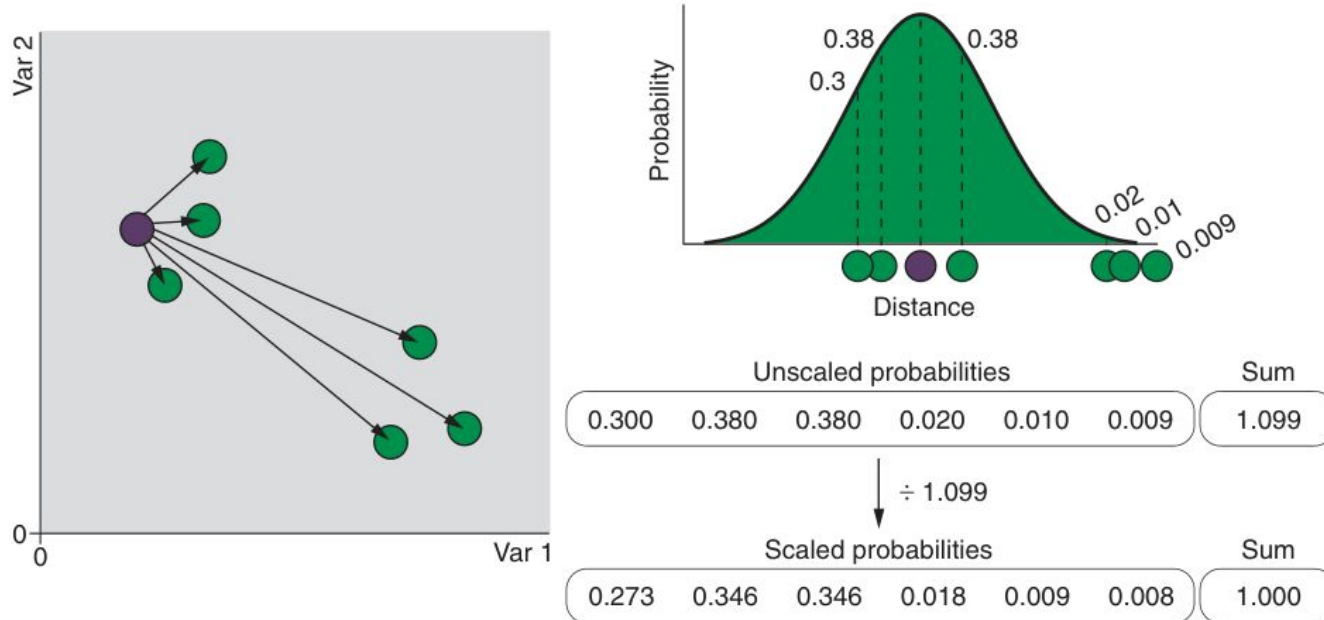


Figure 14.1 t-SNE measures the distance from each case to every other case, converted into a probability by fitting a normal distribution over the current case. These probabilities are scaled by dividing them by their sum, so that they add to 1.

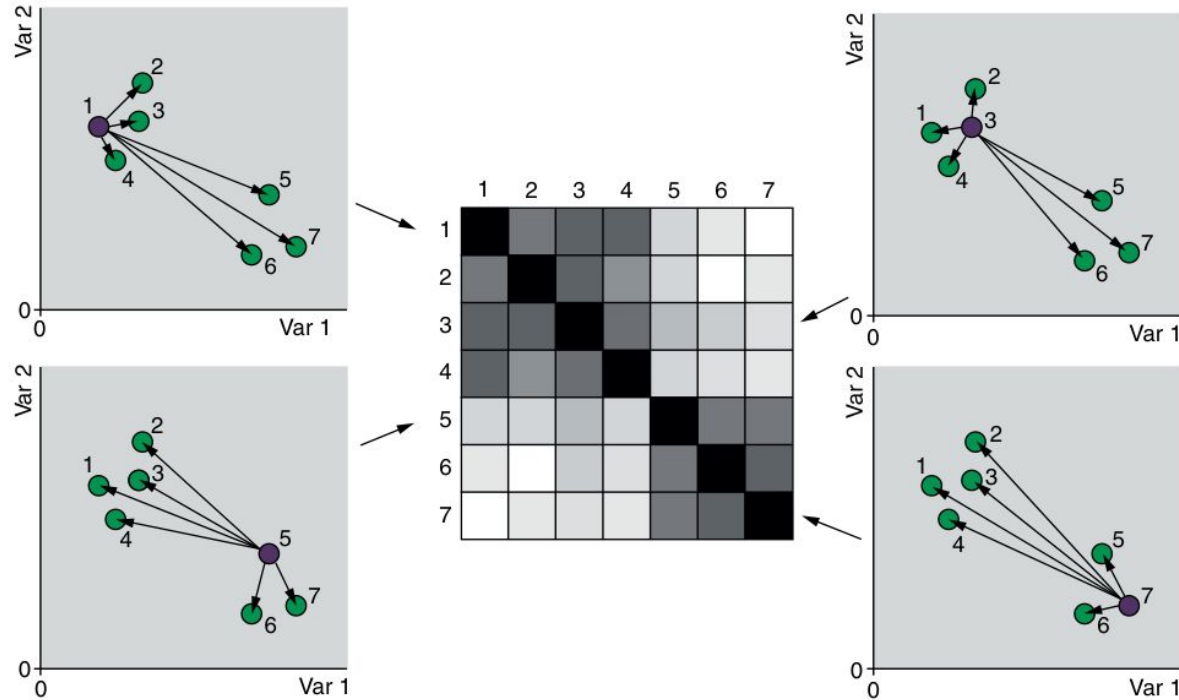


Figure 14.2 The scaled probabilities for each case are stored as a matrix of values. This is visualized here as a heatmap: the closer two cases are, the darker the box is that represents their distance in the heatmap.

t-SNE

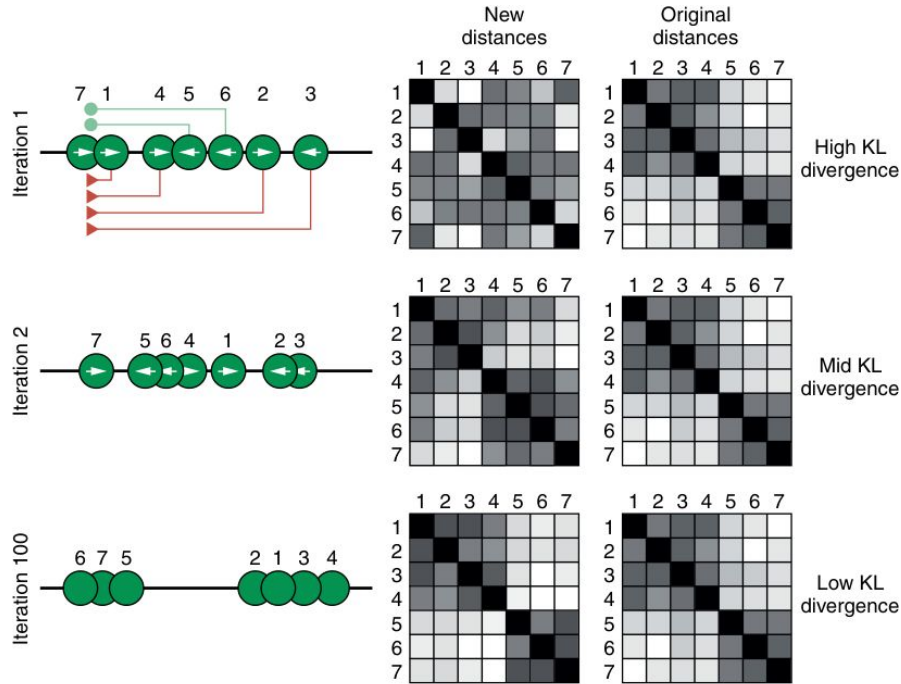


Figure 14.4 Cases are randomly initialized over the new axes (one axis is shown here). The probability matrix is computed for this axis, and the cases are shuffled around to make this matrix resemble the original, high-dimensional matrix by minimizing the Kullback-Leibler (KL) divergence. During shuffling, cases are attracted toward cases that are similar to them (lines with circles) and repulsed away from cases that are dissimilar (lines with triangles).

Distribución t de Student

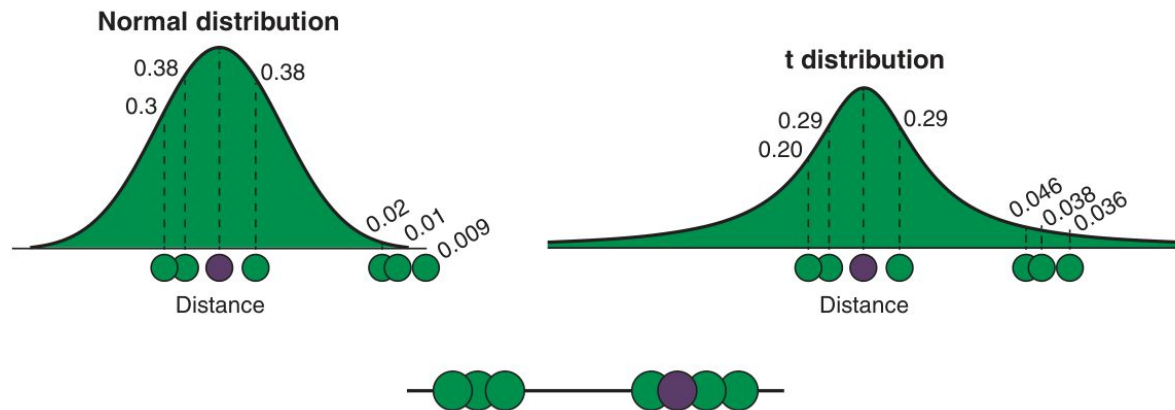


Figure 14.3 When converting distances in the lower-dimensional representation into probabilities, t-SNE fits a Student's t distribution over the current case instead of a normal distribution. The Student's t distribution has longer tails, meaning dissimilar cases are pushed further away to achieve the same probability as in the high-dimensional representation.

La gran novedad es la incorporación de la **distribución t de Student** para modelar los datos en el espacio reducido de baja dimensión.

El algoritmo funciona evaluando las similitudes de los puntos originales usando una curva de distribución normal (Gaussiana) para calcular qué tan probable es que sean vecinos. Sin embargo, cuando intenta ubicar esos mismos puntos en un lienzo 2D, ocurre un fenómeno matemático conocido como el "**problema de aglomeración**" (*crowding problem*). Si t-SNE usara una distribución normal para el gráfico final, todos los clústeres tenderían a amontonarse en el medio del gráfico, haciéndolos imposibles de distinguir.

Al usar una **distribución t** , que tiene un pico más bajo y colas mucho más "gruesas" o altas que una curva normal, el algoritmo logra una mayor separación visual. Esta sutil pero poderosa novedad matemática empuja y separa efectivamente a los puntos que son diferentes, dándoles más espacio en el gráfico de baja dimensión y generando las separaciones nítidas entre clústeres que caracterizan a t-SNE. Para lograr esto, el algoritmo ajusta iterativamente las posiciones minimizando una métrica llamada **divergencia de Kullback-Leibler (KL)**, la cual mide la diferencia entre las probabilidades del espacio original y el nuevo espacio en 2D.

Divergencia Kullback - Leibler

(divergencia KL)

En el contexto del algoritmo t-SNE, la **divergencia de Kullback-Leibler (o divergencia KL)** es la variable matemática clave que se utiliza para medir la diferencia entre dos distribuciones de probabilidad. Tal como comentamos en la definición formal anterior, t-SNE funciona calculando dos matrices o distribuciones de probabilidad:

1. Una distribución que mide las similitudes por pares de los objetos en su espacio original de **alta dimensión**.
2. Una distribución que mide las similitudes por pares de los puntos correspondientes proyectados en el espacio de **baja dimensión** (por ejemplo, en un mapa 2D).

El papel fundamental de la divergencia de Kullback-Leibler es actuar como la función objetivo del algoritmo. Durante el paso de optimización del diseño, t-SNE ajusta iterativamente las posiciones de los puntos en el espacio reducido con el objetivo de **minimizar la divergencia KL entre estas dos distribuciones**.

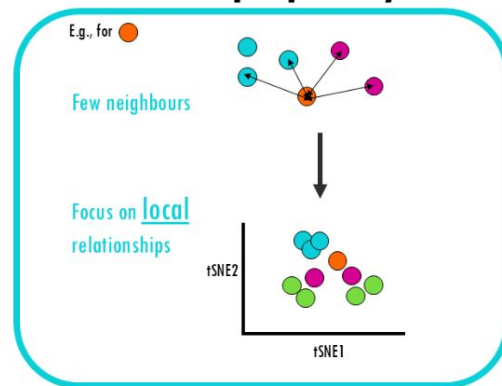
Hiperparámetros de t-SNE

perplexity: controla la amplitud de las distribuciones utilizadas para convertir distancias en probabilidades. Los valores altos se centran más en la estructura global, mientras que los valores bajos se centran más en la estructura local. Los valores típicos oscilan entre 5 y 50. El valor predeterminado es 30.

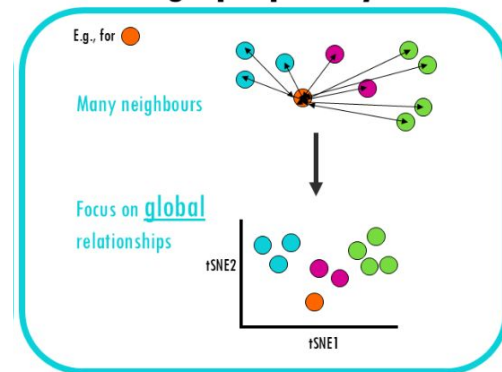
- **Perplejidad baja:** El algoritmo reduce la curva de distribución y considera solo a un número pequeño de vecinos cercanos. Esto hace que t-SNE se enfoque casi exclusivamente en la **estructura local**, creando clusters muy compactos y definidos para objetos que son extremadamente similares.
- **Perplejidad alta:** La distribución Gaussiana se vuelve más ancha y abarca un rango mayor de vecinos. Esto obliga al algoritmo a capturar la **estructura global** del conjunto de datos; si bien los clústeres locales pueden verse menos agrupados o compactos, la disposición general y las relaciones entre los distintos grupos de datos se vuelve mucho más clara.

PERPLEXITY CONTROLS HOW MANY NEIGHBOURS EACH POINT IS CONSIDERING WHEN BUILDING ITS SIMILARITY MAP.

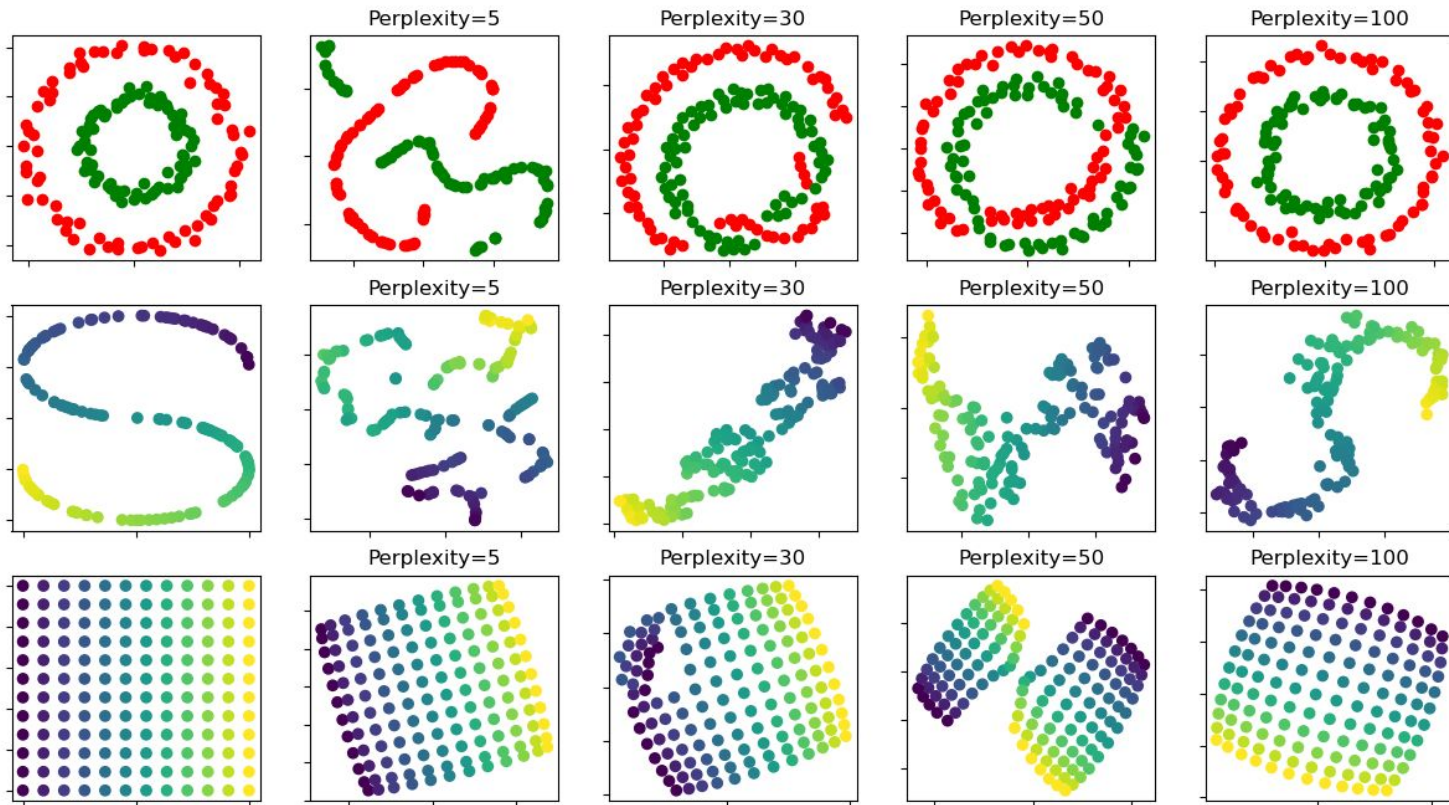
Low perplexity



High perplexity



Hiperparámetro perplexity



max_iter: Número máximo de iteraciones permitidas antes de que finalice el cálculo. Esto dependerá de tu presupuesto computacional, pero es importante contar con suficientes iteraciones para alcanzar la convergencia. El valor predeterminado es 1000.

theta: controla el equilibrio entre velocidad y precisión. Dado que t-SNE es lento, se suele utilizar una implementación llamada t-SNE de Barnes-Hut, que permite realizar la incrustación mucho más rápido, pero con cierta pérdida de precisión. El hiperparámetro θ controla este equilibrio: 0 representa la t-SNE "exacta" y 1 la t-SNE más rápida, pero menos precisa. El valor predeterminado es 0,5.

eta: indica la distancia que recorre cada punto de datos en cada iteración (también llamada tasa de aprendizaje). Los valores más bajos requieren más iteraciones para alcanzar la convergencia, pero pueden dar como resultado una incrustación más precisa. El valor predeterminado es 200, y esto suele ser suficiente.

Algorithm 1: Simple version of t-Distributed Stochastic Neighbor Embedding.

Data: data set $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$,

cost function parameters: perplexity $Perp$,

optimization parameters: number of iterations T , learning rate η , momentum $\alpha(t)$.

Result: low-dimensional data representation $\mathcal{Y}^{(T)} = \{y_1, y_2, \dots, y_n\}$.

begin

 compute pairwise affinities $p_{j|i}$ with perplexity $Perp$ (using Equation 1)

 set $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$

 sample initial solution $\mathcal{Y}^{(0)} = \{y_1, y_2, \dots, y_n\}$ from $\mathcal{N}(0, 10^{-4}I)$

for $t=1$ **to** T **do**

 compute low-dimensional affinities q_{ij} (using Equation 4)

 compute gradient $\frac{\delta C}{\delta \mathcal{Y}}$ (using Equation 5)

 set $\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta \frac{\delta C}{\delta \mathcal{Y}} + \alpha(t) (\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)})$

end

end

PCA	t-SNE
• Tends to compress the visualization • Global (overall) variance is depicted • Deterministic algorithm will produce the same result each run • Does not have hyperparameters to be set • Relatively fast (for datasets that can fit in memory) • Involves linear transformations • Useful as a general dimensionality reduction technique by creating additional principal components	• Tends to cluster the visualization • Local variance is more apparent • Stochastic algorithm introduces randomness into the result • Result can be sensitive to hyperparameters • Relatively slow (but faster approximations exist) • Involves non-linear transformations • Typically used only as a data visualization technique (two or three dimensions)

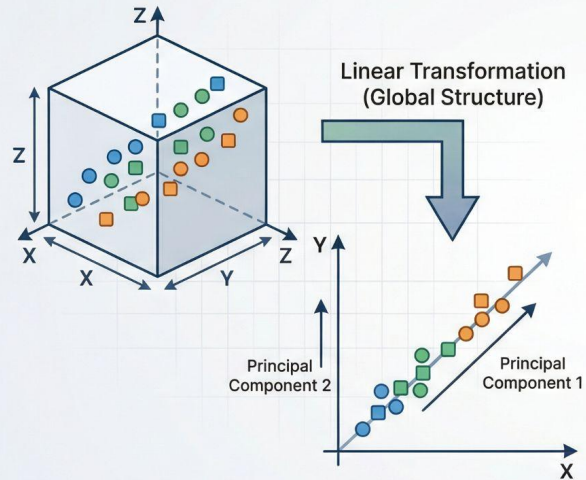
t -SNE vs PCA

	PCA	t -SNE
Purpose	Dimensionality reduction	Data visualisation in low dimensions (typically 2D)
Type of algorithm	Deterministic	Stochastic
Unique solution?	Yes	No
Projection type	Linear	Non-linear
Type of approach	Global	Local/Global*
Interpretation	PCA is just a rotation of axes	Subjective interpretation

**We do not explicitly specify global structure preservation in tSNE.*

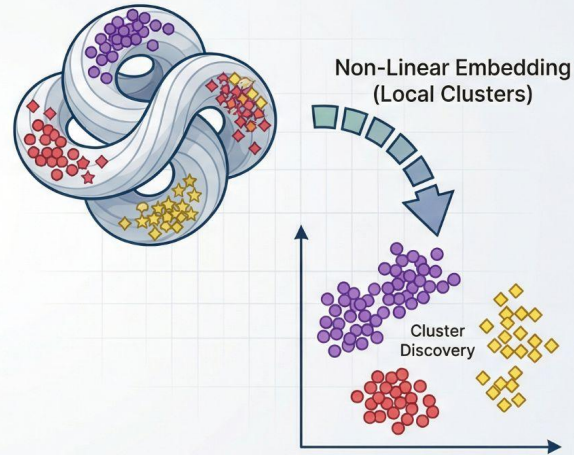
CHOOSING BETWEEN PCA AND t -SNE FOR VISUALIZATION

PCA (Linear Projection)



Preserves Global Structure, Interpretable Axes,
Ideal for Feature Reduction

t -SNE (Non-Linear Clustering)



Reveals Local Clusters, Non-Interpretable Axes,
Ideal for Visual Exploration

Hybrid Approach: Use PCA for Preprocessing (Reduce Dimensions & Noise)
-> Then apply t -SNE for Visualization (Explore Clusters)

Captura estructuras no lineales: A diferencia de PCA, puede mapear geometrías extremadamente complejas (como curvas o variedades retorcidas).

Revela clusters claros: Al enfocarse en las relaciones locales y usar la distribución t , genera separaciones visuales muy nítidas entre grupos de datos.

Maneja múltiples escalas: Gracias al ajuste de la perplejidad, puede adaptarse a densidades variables en los datos.

Alto costo computacional: Su complejidad temporal y espacial suele ser cuadrática, $O(N^2)$, donde N es el número de muestras. No escala bien con millones de filas sin aproximaciones (como Barnes-Hut).

No preserva la estructura global: Las distancias entre clusters distantes en el mapa 2D no tienen un significado real. No podés decir "el cluster A está más cerca del B que del C" basándote solo en el gráfico.

Es estocástico (no determinista): Corre con una semilla aleatoria. Si se ejecuta dos veces con semillas diferentes, el gráfico se verá distinto (aunque los clusters sigan existiendo).

Sensible a los hiperparámetros: El resultado depende críticamente de la **Perplejidad** y del número de iteraciones. Un mal ajuste puede inventar clusters inexistentes o aplastar la estructura.

No genera un modelo de transformación: No se obtiene una función matemática tipo $y = f(x)$ para aplicar a nuevos datos. Si entran datos nuevos, tenés que volver a correr todo el algoritmo con el dataset completo.

UMAP

UMAP (**Uniform Manifold Approximation and Projection**) es una técnica de **reducción de dimensionalidad no lineal** y aprendizaje de variedades (*manifold learning*), diseñada tanto para la visualización de datos complejos como para la reducción general de dimensiones no lineales. Su objetivo es aprender la estructura de la variedad (o *manifold*) subyacente de un conjunto de datos y encontrar una proyección en baja dimensión que preserve la estructura topológica esencial de dicha variedad.

Matemáticamente, el algoritmo se fundamenta en la topología y se basa de forma estricta en **tres suposiciones teóricas clave** sobre los datos:

1. **Los datos están distribuidos uniformemente en una variedad de Riemann** (*Riemannian manifold*).
2. **La métrica riemanniana de dicha variedad es localmente constante** (o puede ser aproximada como tal).
3. **La variedad está localmente conexa.**

A partir de estas tres asunciones, UMAP modela la variedad de los datos originales mediante la construcción de una **estructura topológica difusa** (representada matemáticamente como un complejo simplicial difuso), la cual funciona esencialmente como un grafo de alta dimensión ponderado por probabilidades.

1. Construcción de un grafo de alta dimensión (Fuzzy Simplicial Complex) En este primer paso, UMAP analiza los datos en su espacio original de alta dimensión para entender sus relaciones:

- **Radios locales y vecinos:** UMAP calcula las distancias entre los puntos y dibuja un radio local alrededor de cada uno. Este radio no es fijo para todos los datos, sino que se adapta localmente basándose en la distancia que hay entre el punto y su n -ésimo vecino más cercano (un parámetro llamado **n_neighbors**).
- **Conexiones "difusas":** Se conectan los puntos cuyos radios se superponen, pero la probabilidad o "fuerza" de esa conexión disminuye a medida que la distancia entre los puntos crece.
- **Garantía de conexión:** Para evitar que queden puntos aislados o se fragmente la información local, UMAP garantiza que cada punto esté fuertemente conectado, como mínimo, con su vecino más cercano.

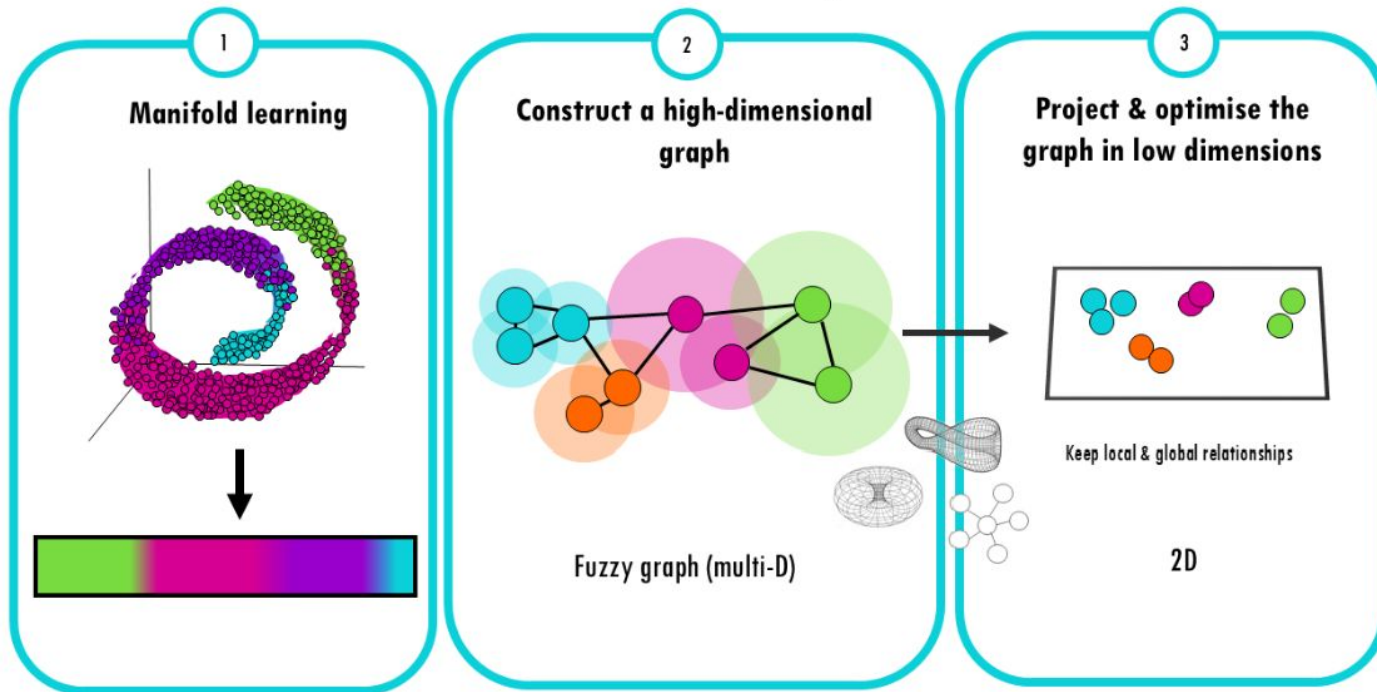
- **Simetría:** Dado que la cercanía es relativa al punto de vista de cada dato, la relación no siempre es bidireccional desde el inicio. UMAP usa una operación matemática (unión difusa) para promediar estas diferencias y hacer que las puntuaciones de similitud sean simétricas, consolidando un único grafo de alta dimensión.

2. Inicialización en baja dimensión Antes de acomodar los datos en 2D o 3D, UMAP debe colocarlos en un lienzo de baja dimensión. A diferencia de otros métodos (como t-SNE) que inician colocando los puntos de manera completamente aleatoria, UMAP los inicializa utilizando una técnica matemática llamada **incrustación espectral** (*spectral embedding*). Esto permite que el algoritmo arranque desde una aproximación base que tiene sentido lógico, logrando que el diseño sea más consistente y rápido.

3. Optimización del diseño en baja dimensión (Graph Layout) El último paso consiste en organizar este nuevo grafo para que se parezca lo más posible al grafo de alta dimensión que creamos en el primer paso.

- **Atracción y repulsión:** UMAP ajusta las posiciones moviendo ligeramente los puntos iterativamente. Si dos puntos son similares (vecinos en la alta dimensión), los mueve más cerca el uno del otro para maximizar su similitud; si son diferentes, los empuja para alejarlos.
- **Similitud geométrica:** Para estimar esta similitud en el nuevo espacio reducido, UMAP utiliza una curva fija en forma de campana derivada de una distribución t de Student, similar a cómo lo hace t-SNE.
- **Minimización del error:** Para guiar estos movimientos, el algoritmo utiliza un método de optimización llamado *descenso de gradiente estocástico* (Stochastic Gradient Descent). Así, mueve los puntos paso a paso minimizando una función de "entropía cruzada", la cual penaliza las diferencias estructurales entre el grafo de alta y baja dimensión.

How does UMAP summarise many dimensions into 2?



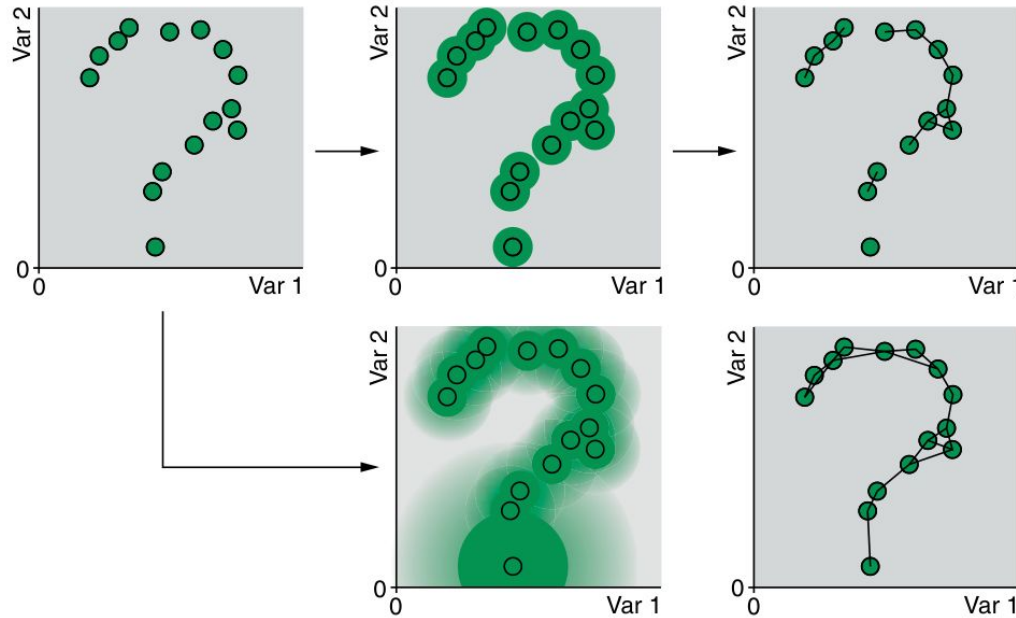


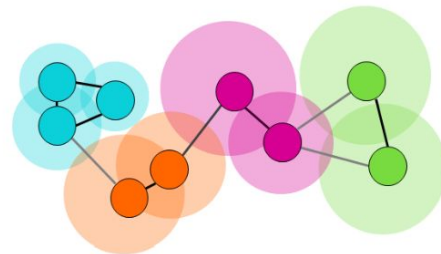
Figure 14.8 How UMAP learns a manifold. UMAP expands a search region around each case. A naive form of this is shown in the top row, where the radius of each search region is the same. When cases with overlapping search regions are connected by edges, there are gaps in the manifold. In the bottom row, the search region extends to the nearest neighbor and then extends outward in a fuzzy way, with a radius inversely related to the density of data in that region. This results in a complete manifold.

Hiperparámetros de UMAP

Los principales hiperparámetros de UMAP que controlan cómo el algoritmo modela y proyecta los datos son cuatro:

- **n_neighbours** (número de vecinos): Este parámetro **controla el equilibrio entre la preservación de la estructura local y la estructura global** de los datos. Define el tamaño del vecindario local que UMAP analiza para aprender la estructura original.
 - Un valor bajo hace que el algoritmo se centre en los detalles muy locales, lo que preserva las relaciones estrechas y crea agrupaciones (clusters) muy definidas.
 - Un valor alto obliga a UMAP a considerar una porción más amplia de los datos, sacrificando el detalle local fino para capturar mejor las relaciones globales y la organización general del conjunto de datos.

n_neighbours = 2



n_neighbours = 4



The hyperparameter **n_neighbours** is used to control the size of the radii

Hiperparámetros de UMAP

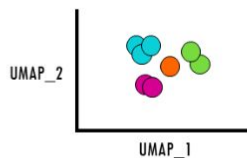
- **min_dist** (distancia mínima): Controla **qué tan cerca pueden agruparse los puntos en el espacio reducido**. Físicamente, establece la distancia mínima permitida entre puntos en la proyección final.
 - Un valor bajo (por ejemplo, cercano a 0) permite que los puntos se empaqueten densamente, lo cual es ideal si te interesa el clustering o identificar estructuras locales finas.
 - Un valor más alto dispersa los puntos, evitando que se amontonen y ofreciendo una visión más amplia de la estructura topológica global.

min_dist is the minimum distance between points in the low-dimensional space.

<https://biostatsquid.com/umap-simply-explained/>

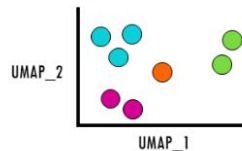
Low min_dist

Focus on local relationships



High min_dist

Focus on global relationships



- **n_components** (número de componentes): Determina la cantidad de dimensiones del nuevo espacio reducido al que se proyectarán los datos. Aunque lo más común es reducir los datos a 2 o 3 dimensiones para poder visualizarlos en gráficos, UMAP escala muy bien y permite proyectar a más dimensiones (por ejemplo, 10 o 50) si el objetivo posterior es aplicar técnicas de machine learning o clustering basados en densidad.
- **metric** (métrica de distancia): Define cómo se calcula la distancia entre los puntos en el espacio original de alta dimensión. Por defecto, UMAP soporta una gran variedad de métricas espaciales y angulares (como Euclidiana, Manhattan, Coseno, Mahalanobis, etc.), así como métricas para datos binarios. Además, permite la creación de métricas personalizadas definidas por el usuario.
- **set_op_mix_ratio**: controla matemáticamente cómo el algoritmo combina los diferentes conjuntos locales (los vecindarios alrededor de cada punto de datos) para consolidar un único grafo global. Específicamente, este parámetro define la interpolación entre una operación de "unión" y una de "intersección" al momento de mezclar las probabilidades de conexión entre los puntos. **Valor de 1.0 (por defecto)**: Representa una operación de **unión pura**. Este es el enfoque estándar porque evita que el grafo resultante se rompa en múltiples componentes desconectados, logrando una representación de alta calidad que retiene mejor la estructura global y no local de los datos

Ventajas y desventajas t-SNE y UMAP

The strengths of t-SNE and UMAP are as follows:

- They can learn nonlinear patterns in the data.
- They tend to separate clusters of cases better than PCA.
- UMAP can make predictions on new data.
- UMAP is computationally inexpensive.
- UMAP preserves both local *and* global distances.

The weaknesses of t-SNE and UMAP are these:

- The new axes of t-SNE and UMAP are not directly interpretable in terms of the original variables.
- t-SNE cannot make predictions on new data (different result each time).
- t-SNE is computationally expensive.
- t-SNE doesn't necessarily preserve global structure.
- They cannot handle categorical variables natively.

- **Mayor velocidad y escalabilidad:** UMAP es significativamente más rápido que t-SNE y escala muy bien tanto con grandes cantidades de datos como con una alta cantidad de dimensiones.
- **Mejor preservación de la estructura global:** Mientras que t-SNE se enfoca fuertemente en las agrupaciones locales a expensas del panorama general, la gran ventaja matemática de UMAP es que logra un excelente equilibrio .
- **Versatilidad en el número de dimensiones de salida:** Mientras que t-SNE está principalmente diseñado para visualizaciones en 2D o 3D, UMAP escala excepcionalmente bien a dimensiones superiores
- **Parámetros más intuitivos y soporte avanzado de métricas:** Los hiperparámetros de UMAP (como **n_neighbors** y **min_dist**) brindan a los usuarios un control mucho más comprensible sobre qué tanto desean priorizar la vista local versus la global, y qué tan compactos deben verse los clústeres

- **Falta de interpretabilidad directa (tamaños y distancias):** Las dimensiones del espacio reducido de UMAP no tienen un significado analítico o físico específico. Además, en los gráficos resultantes, los tamaños relativos de los clústeres no significan nada. De igual manera, las distancias entre diferentes clústeres pueden carecer de significado real y no son directamente interpretables de la misma manera que lo serían en algoritmos lineales como PCA
- **Falta de reproducibilidad exacta (Naturaleza estocástica):** UMAP es un algoritmo no determinista (estocástico) debido a que utiliza inicializaciones aleatorias en su proceso de optimización. Esto significa que ejecutar el algoritmo varias veces sobre los mismos datos puede arrojar resultados visualmente distintos
- **Sensibilidad a los hiperparámetros:** El gráfico que obtienes cambiará significativamente dependiendo de los valores que asignes a los parámetros principales, como **n_neighbors** y **min_dist**. Debido a esto, UMAP requiere de experimentación y optimización para encontrar la configuración correcta.

- **Creación de agrupaciones espurias a partir del ruido:** Debido a su base topológica, UMAP puede tender a forzar el hallazgo de una estructura de variedad incluso dentro del ruido de un conjunto de datos.
- **Incapacidad para separar clústeres anidados:** UMAP tiene una falla documentada cuando se enfrenta a un clúster denso y compacto que está contenido dentro de un clúster más amplio y disperso.
- **Pérdida de la geometría global exacta:** Aunque hace un mejor trabajo que t-SNE preservando el panorama general, UMAP sigue priorizando la topología local por encima de la geometría global.

Problemas comunes en UMAP

1. Elegir los parámetros adecuados.
2. Rendimiento computacional en grandes conjuntos de datos.
3. Interpretación de distancias en proyecciones de UMAP.
4. Problemas de reproducibilidad.
5. Valores atípicos que distorsionan la visualización.

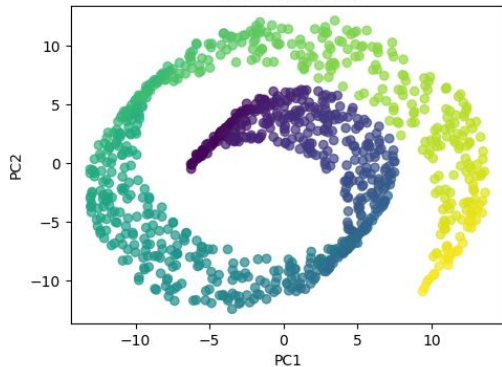
Para una descripción mas detallada de estos puntos, visitar:

<https://www.datacamp.com/es/tutorial/understanding-umap-guide-to-dimensionality-reduction>

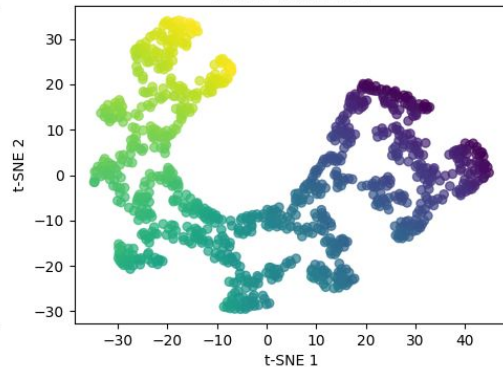
Comparativa

Comparison of Dimensionality Reduction Methods

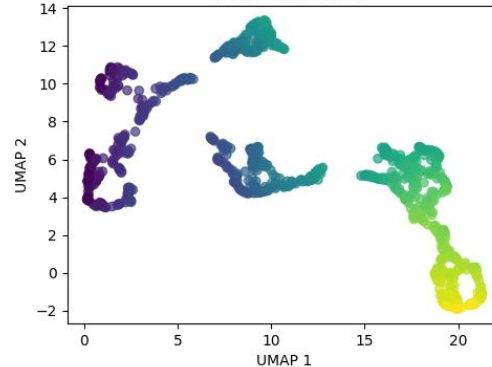
PCA - Swiss Roll



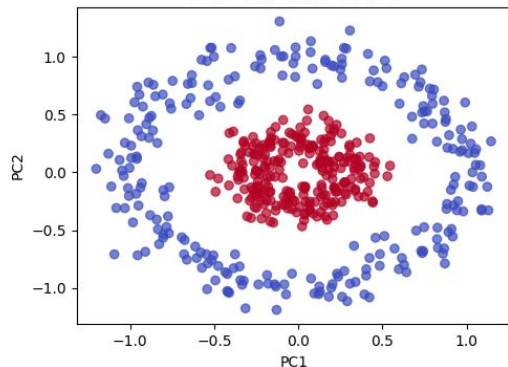
t-SNE - Swiss Roll



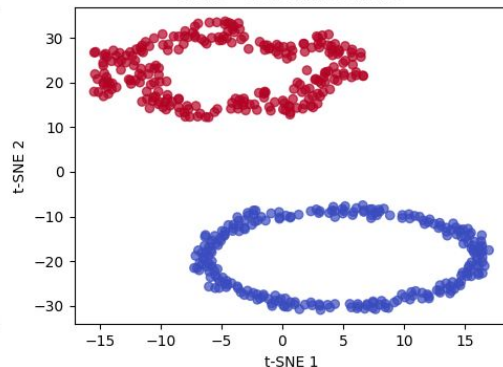
UMAP - Swiss Roll



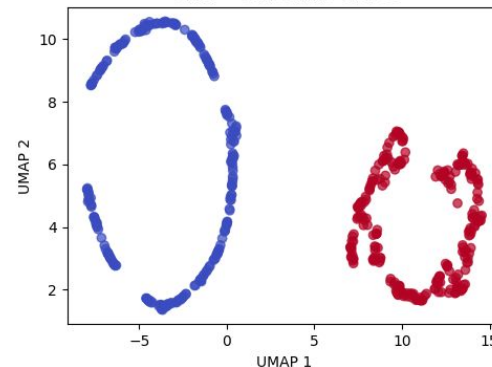
PCA - Concentric Circles



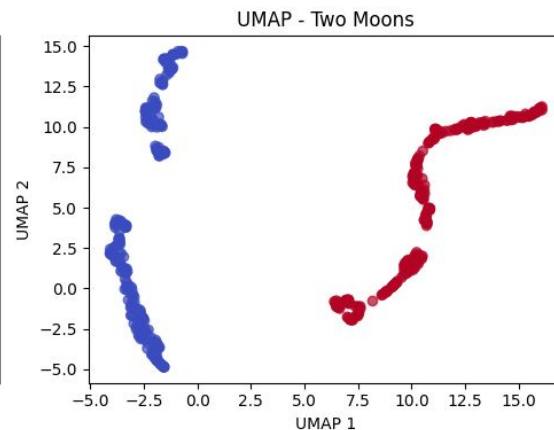
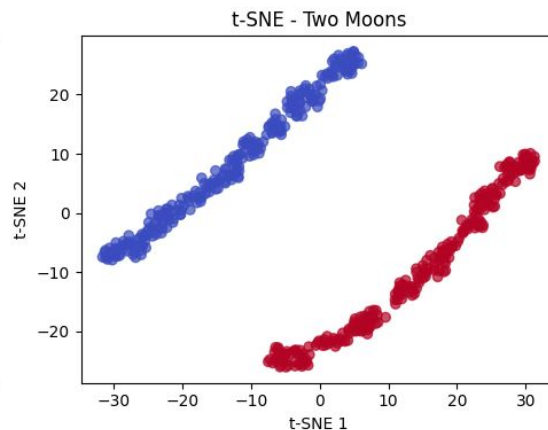
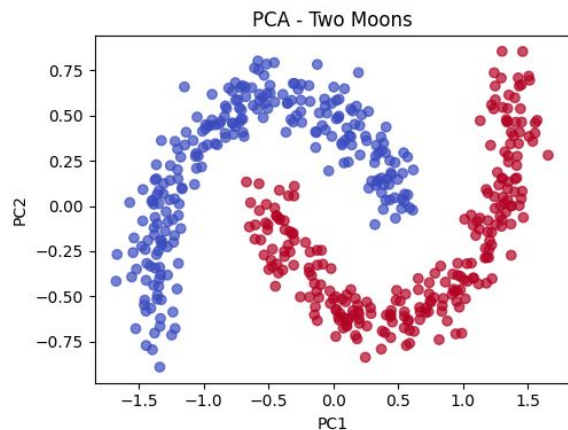
t-SNE - Concentric Circles



UMAP - Concentric Circles

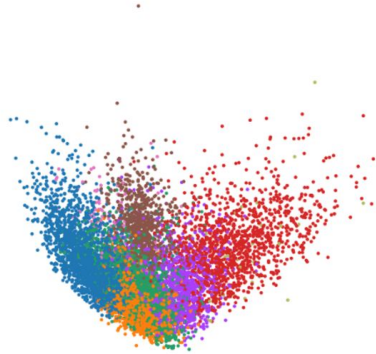


Comparativa



Comparativa

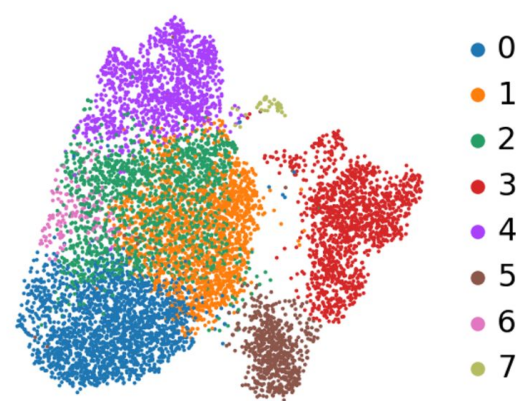
PCA



tSNE



UMAP



PCA	t-SNE	UMAP
Linear	Non-linear	Non-linear
Deterministic	Non-deterministic	Non-deterministic
Computationally cheap	Computationally expensive	Computationally efficient

	PCA	t-SNE	UMAP
Type of Method	Linear method (uses linear transformations)	Non-linear method (focuses on local structure)	Non-linear method (focuses on both local and global structure)
Focus	Maximizing variance (global structure)	Preserving local structure (neighborhoods)	Preserving both local and global structure
Preserves	Variance (overall spread of the data)	Local relationships (similarity between neighbors)	Local and global relationships (overall shape and clusters)
Output	Linear transformation of the data into principal components	2D or 3D representation that reflects local similarities	2D or 3D representation with more global structure
Scalability	Highly scalable (works well with large datasets)	Computationally expensive on large datasets	Scalable, faster than t-SNE, works well with large datasets
Speed	Fast	Slow, especially for large datasets	Faster than t-SNE, more scalable
Reproducibility	Very stable and deterministic	Results can vary with different runs	More stable than t-SNE, but less deterministic than PCA
Interpretability	Results are easy to interpret (principal components)	Results are harder to interpret (abstract relationships)	Results are more interpretable than t-SNE, but not as clear as PCA
Best for	When you want to preserve overall variance and reduce dimensionality linearly (e.g., for feature extraction, noise reduction)	When you want to explore local structure and identify clusters in the data	When you want to preserve both local and global structure, especially in complex, large datasets

- PCA
<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
- Kernel PCA
<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.KernelPCA.html>
- t-SNE
<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>
- UMAP https://umap-learn.readthedocs.io/en/latest/basic_usage.html
- Paper t-SNE
<https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>
- Paper UMAP <https://arxiv.org/pdf/1802.03426>

Aplicaciones Espaciales

Soil fertility quality assessment based on geographically weighted principal component analysis (GWPCA) in large-scale areas

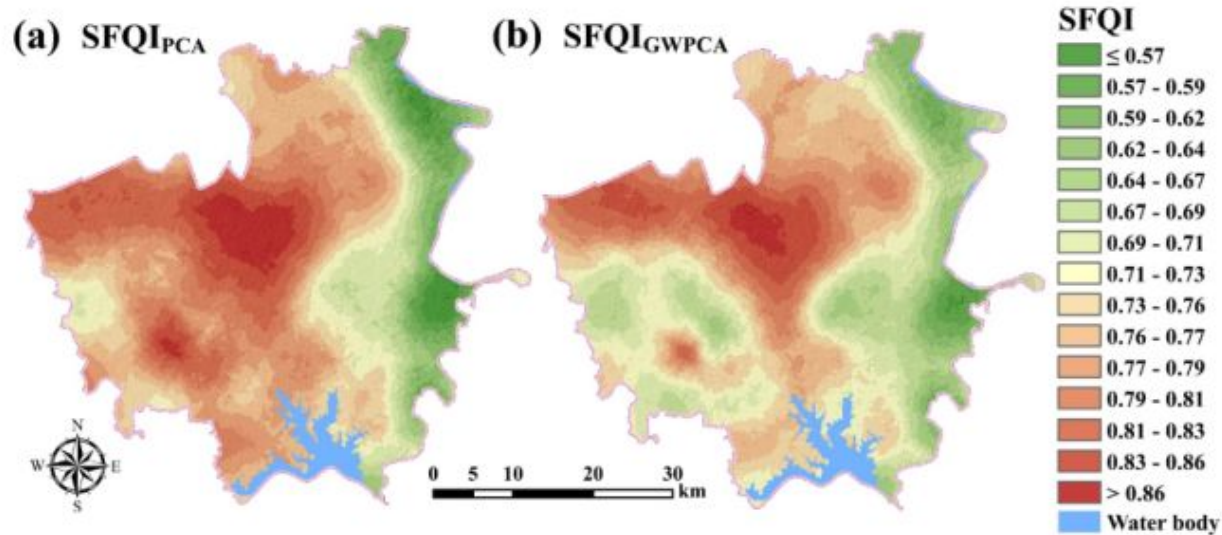


Fig. 8. Spatial distribution of (a) SFQI_{PCA} and (b) SFQI_{GWPCA} in Shayang County, China. SFQI_{PCA}, soil fertility quality index calculated by principal component analysis; SFQI_{GWPCA}, soil fertility quality index calculated by geographically weighted principal component analysis.

A geographically weighted regression model augmented by Geodetector analysis and principal component analysis for the spatial distribution of PM_{2.5}

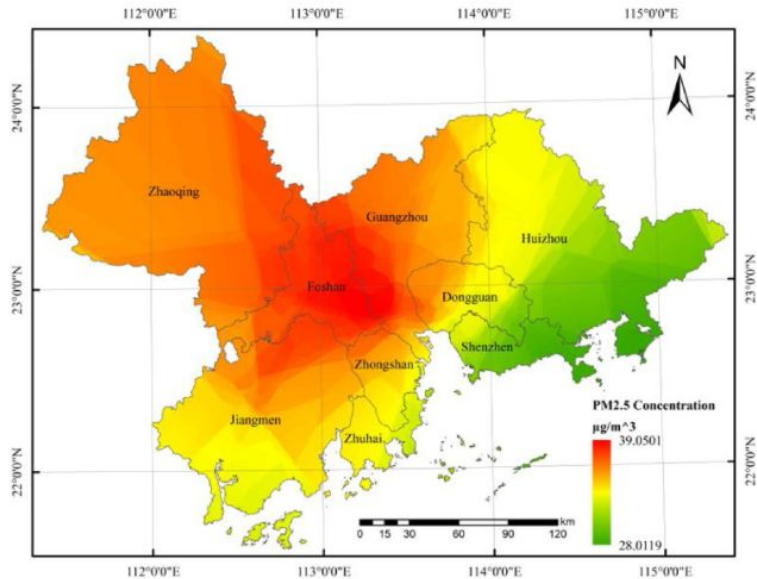


Fig. 3. Spatial distribution of PM_{2.5} concentrations based on an augmented GWR model.

<https://www.sciencedirect.com/science/article/abs/pii/S2210670720300937>

Using *t*-distributed Stochastic Neighbor Embedding (*t*-SNE) for cluster analysis and spatial zone delineation of groundwater geochemistry data

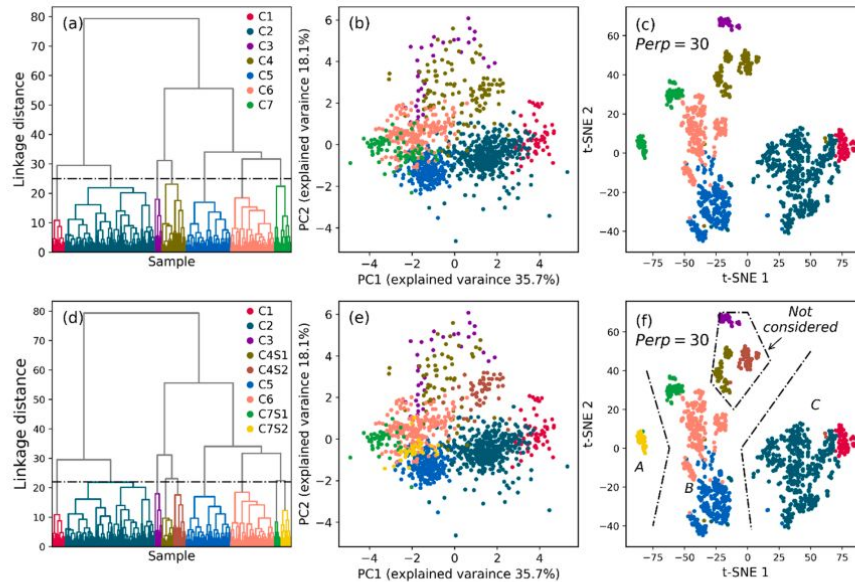
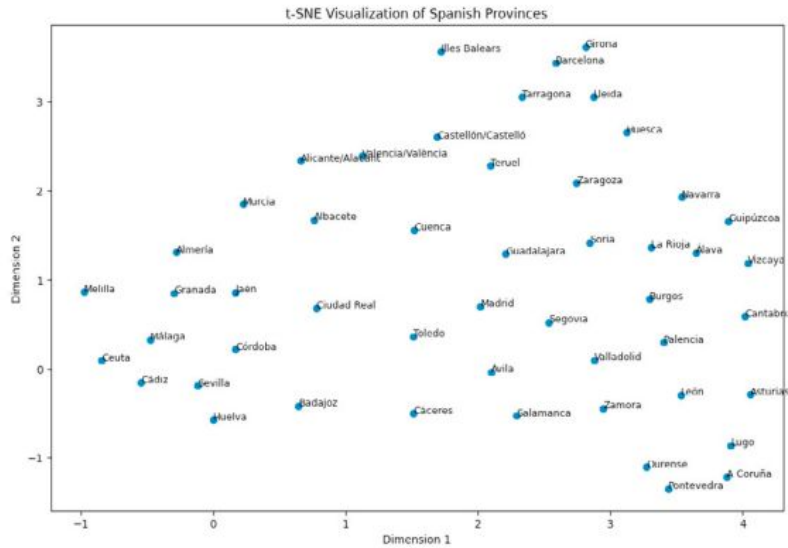


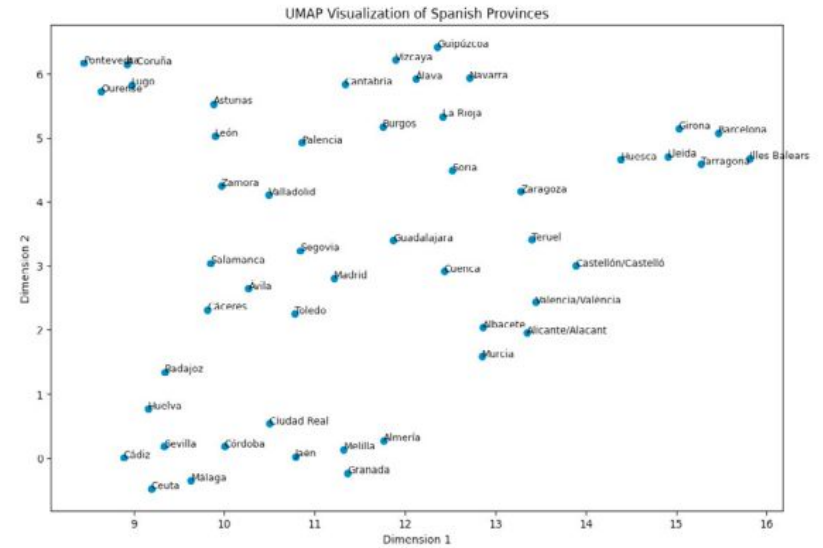
Fig. 7. (a) HCA dendrogram using the Ward method with Euclidian distance with a phenon line location at a linkage distance of 25, which yielded seven clusters, (b) two-dimensional PCA visualization of the seven clusters, and (c) two-dimensional *t*-SNE visualization of the seven clusters. (d)–(f) are equivalent to (a)–(c), respectively, except that there were nine clusters with the phenon line at 22. Each point in the PCA and *t*-SNE visualizations represents one sample.

<https://www.sciencedirect.com/science/article/abs/pii/S0022169421001931>

UMAP for geospatial data visualization



(a) t-SNE best fit run from 100 tries for Spanish province placement.



(b) UMAP best fit run from 100 tries for Spanish province placement.

Bibliografía

- *Machine Learning with R, the tidyverse and mlr*
- *Kernel PCA*
<https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-kernel-pca/>
- <https://www.baeldung.com/cs/kernel-principal-component-analysis>
- GIF: <https://acortar.link/KePkqj>
- Blog de ciencia de datos
https://cienciadedatos.net/documentos/35_principal_component_analysis#Introducci%C3%B3n
- PCA (Udacity) <https://www.youtube.com/watch?v=kW9R0nD69OU>
- Easy UMAP <https://biostatsquid.com/umap-simply-explained/>
- Easy t-SNE <https://biostatsquid.com/easy-t-sne-explained-with-an-example/>
- Documentación de UMAP: <https://umap-learn.readthedocs.io/en/latest/index.html>
- tSNE tutorial datacamp <https://www.datacamp.com/es/tutorial/introduction-t-sne>
- Como utilizar t-SNE de manera efectiva <https://distill.pub/2016/misread-tsne/>